

```
package com.olal00.cyx.data.domain.common;
import com.olal00.cyx.common.CyxConst;
import com.olal00.cyx.data.domain.DomainBase;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;

public class Book extends DomainBase<Book, String> {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    @Field("parent_id")
    private String parentId; //父节点 ID
    @Field("parent_name")
    private String parentName; //上一级名称
    @Field("level")
    private Integer level; //EnumBookLevelType 等级 1~5
    @Field("subject_id")
    private Integer subjectId; //学科 id
    @Field("subject_level")
    private Integer subjectLevel; //学科阶段 EnumSubjectLevel 1:小学 2:初中 3:高中
    @Field("name")
    private String name; //名称
    @Field("short_name")
    private String shortName; //level2 专有 简称
    @Field("pic_url")
    private String picUrl;
    @Field("grade")
    private Integer grade; //年级 level1 没有年级
    @Field("rank")
    private Integer rank; //排序, 正序
    @Field("status")
    private String status; //教材状态 ONLINE|OFFLINE|SOONLINE
    @Field("exercise_status")
    private String exerciseStatus; //练习状态 ONLINE|OFFLINE|SOONLINE
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import com.olal00.tnzb.data.domain.form.BookCatalog;
import com.olal00.tnzb.data.domain.form.QuestionAnswer;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.util.List;
/**
@Data
```

```
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("book_level_log")
public class BookLevelLog extends DomainBase<BookLevelLog, String> {
    @Field("user_id")
    private String userId; //用户 ID
    @Field("question_group_id")
    private String questionGroupId; //题目分组 id
    @Field("book_catalog")
    private BookCatalog bookCatalog;
    @Field("level")
    private Integer level; //第几关
    @Field(name = "star")
    private Double star; //几颗星 0|0.5|1|1.5|2|2.5|3
    @Field("power")
    private Integer power; //奖励精力值
    @Field("right_count")
    private Integer rightCount; //答对几题
    @Field("use_prop_count")
    private Integer usePropCount; //使用道具次数
    @Field("relive_count")
    private Integer reliveCount; //使用复活次数
    @Field("question_answers")
    private List<QuestionAnswer> questionAnswers; //答题结果
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.common.TnzbConst;
import com.olal00.tnzb.data.domain.DomainBase;
import com.olal00.tnzb.data.domain.form.BookCatalog;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.util.List;
/**
@Data
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("book_question")
public class BookQuestion extends DomainBase<BookQuestion, String> {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    @Field("group_order_num")
    private Integer groupOrderNum; //第几关
    @Field("group_id")
    private String groupId; //分组 id, 就是第几关
    @Field("type")
```

```
private String type; //类别: 学科
@Field("difficulty")
private Integer difficulty; //EnumQuestionDifficulty 难度: 容易、较易、中、较难、难
@Field("is_super")
private Integer isSuper; //是否终极一题
@Field("order_num")
private Integer orderNum; //序号
//题目、选项、答案、解析
@Field("title")
private String title; //题干
@Field("img_url")
private String imgUrl; //题目图片, 可以没有
@Field("options")
private List<String> options; //选项, 都是纯文字, 10 个字符以内, 最多四个
@Field("answer")
private String answer; //答案 ABCD, 0=A, 1=B, 2=C, 3=D
@Field("answer_index")
private Integer answerIndex; //答案从 0 开始, 0=A, 1=B, 2=C, 3=D
@Field("analysis")
private String analysis; //解析
@Field("book_catalog")
private BookCatalog bookCatalog;
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.time.LocalDate;
/**
 * 签到记录表
 */
@Data
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("sign_in_daily_record")
public class SignInDailyRecord extends DomainBase<SignInDailyRecord, String> {
    /**
     * 用户 id
     */
    @Field("user_id")
    private String userId;
    /**
     * 签到日期
     */
    @Field("sign_in_date")
```

```
private LocalDate signInDate;
/**
 * 周几 1 ~ 7
 */
@Field("day_of_week")
private Integer dayOfWeek;
/**
 * 期数, xxxx 年 xx 周
 */
@Field("week")
private Integer week;
/**
 * 标题
 */
@Field("title")
private String title;
/**
 * 奖励的金币或道具数量
 */
@Field(name = "award_value")
private Integer awardValue;
/**
 * EnumSignAwardType 奖励的类型, 10:金币; 3:复活道具
 */
@Field("award_type")
private Integer awardType;
/**
 * 是否双倍
 */
@Field("is_double")
private Integer isDouble;
/**
 * 当周签到了几次
 */
//@Field("week_attendance_days")
//private Integer weekAttendanceDays;
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
/**
 * 签到配置表
 */
@Data
@Accessors(chain = true)
```

```
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("sign_in_settings")
public class SignInSettings extends DomainBase<SignInSettings, String> {
    /**
     * 标题
     */
    @Field("title")
    private String title;
    /**
     * 奖励的金币或道具数量
     */
    @Field(name = "award_value")
    private Integer awardValue;
    /**
     * EnumSignAwardType 奖励的类型, 10:金币; 3:复活道具
     */
    @Field("award_type")
    private Integer awardType;
    /**
     * 奖励说明
     */
    @Field("award_desc")
    private String awardDesc;
    /**
     * 双倍奖励说明
     */
    @Field("award_desc2")
    private String awardDesc2;
    /**
     * 周几 1 ~ 7
     */
    @Field("day_of_week")
    private Integer dayOfWeek;
    /**
     * 图标
     */
    @Field("icon")
    private String icon;
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
/**
 * 签到配置表
```

```
*/
@Data
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("task_base")
public class TaskBase extends DomainBase<TaskBase, String> {
    /**
     * 图标
     */
    @Field("icon")
    private String icon;
    /**
     * 标题
     */
    @Field("title")
    private String title;
    /**
     * 副标题
     */
    @Field("award_desc")
    private String awardDesc;
    /**
     * 奖励的金币或道具数量
     */
    @Field(name = "award_value")
    private Integer awardValue;
    /**
     * 任务类型 EnumTaskType
     */
    @Field(name = "task_type")
    private Integer taskType;
    /**
     * EnumSignAwardType 奖励的类型，10:金币；3:复活道具
     */
    @Field("award_type")
    private Integer awardType;
    /**
     * 奖品图标
     */
    @Field("award_icon")
    private String awardIcon;
    /**
     * 奖励说明
     */
    @Field("award_desc2")
    private String awardDesc2;
    /**
     * 需要完成次数
     */
}
```

```
@Field("need_count")
private Integer needCount;
/**
 * 序号
 */
@Field("order_num")
private Integer orderNum;
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.time.LocalDate;
/**
 * 签到记录表
 */
@Data
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("task_record")
public class TaskRecord extends DomainBase<TaskRecord, String> {
    /**
     * 用户 id
     */
    @Field("user_id")
    private String userId;
    /**
     * 任务日期
     */
    @Field("task_date")
    private LocalDate taskDate;
    /**
     * 任务 ID
     */
    @Field("task_id")
    private String taskId;
    /**
     * 标题
     */
    @Field("title")
    private String title;
    /**
     * 奖励的金币或道具数量
     */
    @Field(name = "award_value")
```

```

        private Integer awardValue;
    /**
     * EnumSignAwardType 奖励的类型, 10:金币; 3:复活道具
     */
    @Field("award_type")
    private Integer awardType;
}

package com.olal00.tnzb.data.domain.common;
import com.olal00.tnzb.data.domain.DomainBase;
import com.olal00.tnzb.data.domain.form.QuestionAnswer;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.ToString;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;
import java.util.List;
/**
@Data
@Accessors(chain = true)
@EqualsAndHashCode(callSuper = true)
@ToString(callSuper = true)
@Document("up_level_log")
public class UpLevelLog extends DomainBase<UpLevelLog, String> {
    @Field("user_id")
    private String userId;//用户 ID
    @Field("question_group_id")
    private String questionGroupId; //题目分组 id
    @Field("level")
    private Integer level; //第几关
    @Field(name = "star")
    private Double star; //几颗星 0|0.5|1|1.5|2|2.5|3
    @Field("power")
    private Integer power;//奖励精力值
    @Field("right_count")
    private Integer rightCount; //答对几题
    @Field("use_prop_count")
    private Integer usePropCount; //使用道具次数
    @Field("relive_count")
    private Integer reliveCount; //使用复活次数
    @Field("question_answers")
    private List<QuestionAnswer> questionAnswers; //答题结果
}

package com.olal00.tnzb.data.domain.form;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Field;
import java.io.Serializable;
/**

```


@Data

@Accessors(chain = true)

```
public class BookCatalog implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    @Field("subject_id")
    private Integer subjectId;
    @Field("subject_level")
    private Integer subjectLevel; //学科阶段 EnumSubjectLevel 1:小学 2:初中 3:高中
    @Field("grade")
    private Integer grade; //年级
    @Field("series_id")
    private String seriesId;
    @Field("book_id")
    private String bookId;
    @Field("chapter_id")
    private String chapterId;
    @Field("section_id")
    private String sectionId;
    @Field("small_section_id")
    private String smallSectionId;
    @Field("series_name")
    private String seriesName;
    @Field("book_name")
    private String bookName;
    @Field("short_name")
    private String shortName; //book 简称
    @Field("chapter_name")
    private String chapterName;
    @Field("section_name")
    private String sectionName;
    @Field("small_section_name")
    private String smallSectionName;
}
```

<!--

@desc 金币记录

-->

<template>

<div class="content-frame">

<!-- 筛选 -->

<div class="paper-form">

<el-form :inline="true">

<el-form-item label="用户 ID:" style="width:260px;">

<el-input clearable v-model="query.filter.is_str_userId" placeholder="用户 ID"></el-input>

</el-form-item>

<el-form-item label="时间:" style="width: 430px">

<el-date-picker v-model="query.timeRang" type="datetimerange" value-format="YYYY-MM-DD HH:mm:ss"

start-placeholder="开始时间" end-placeholder="结束时间" :shortcuts="shortcuts" @change="timeRangChange">

```

        </el-date-picker>
      </el-form-item>
      <el-form-item>
        <el-button type="primary" @click="querySubmit" :loading="queryLoading"> 查询
      </el-button>
    </el-form-item>
  </el-form>
</div>
<!-- 列表 -->
<div class="bigContent">
  <div class="itemArr">
    <el-table :data="list" height="100%" v-loading="queryLoading"
      stripe :header-cell-style="{ background: '#fafafa' }">
      <el-table-column label="用户 ID" prop="userId"></el-table-column>
      <el-table-column label="金币" prop="goldCoin"></el-table-column>
      <el-table-column label="剩余金币" prop="usableGoldCoin"></el-table-column>
      <el-table-column label="说明" prop="recordDesc"></el-table-column>
      <el-table-column label="时间" prop="createdAt"></el-table-column>
    </el-table>
  </div>
<!-- 分页 -->

<pagination :total="query.total" :currentPage="query.pageNum" :defaultPageSize="query.
  pageSize"
  @pageChange="pageChange" @pageSizeChange="pageSizeChange"></pagination>
</div>
</div>
</template>
<script lang="ts">
import { defineComponent, onMounted, reactive, ref } from "vue";
import pagination from "@components/pagination.vue";
import { ElMessage } from "element-plus";
import { findGoldCoinRecordList } from "../../tools/log";
import { timeformat, addDays, timeSelect } from "../../tools/dateTool";
export default defineComponent({
  name: "goldCoinRecord",
  components: {
    pagination,
  },
  setup() {
    const shortcuts = timeSelect();
    const timeRangChange = (data: any) => {
      if(data == null) {
        ElMessage.warning("时间不能为空");
        return false;
      }
      const oDate1: any = new Date(data[0]).getTime();
      const oDate2: any = new Date(data[1]).getTime();
      console.log('oDate2 - oDate1', oDate2 - oDate1)
      const oneDay = 24 * 3600 * 1000;

```

```
    if (query.filter.is_str_userId && query.filter.is_str_userId.length > 8) {
      if (oDate2 - oDate1 > oneDay * 30) {
        ElMessage.warning("有用户 ID 的时间范围不能大于 30 天");
        query.timeRang[1] = timeformat(new Date(oDate1 + oneDay * 30));
        return false;
      }
    } else if (oDate2 - oDate1 > oneDay) {
      ElMessage.warning("没有用户 ID 的时间范围不能大于 1 天");
      query.timeRang[1] = timeformat(new Date(oDate1 + oneDay));
      return false;
    }
    return true;
  };
const list = ref<any[]>([]);
const queryLoading = ref<boolean>(false);
const query = reactive<any>({
  total: 0,
  pageNum: 1,
  pageSize: 20,
  sortField: "createdAt",
  sortOrder: "desc",
  timeRang: [timeformat(addDays(0)), timeformat(addDays(1))],
  filter: {},
});
// 查询列表
const queryList = () => {
  if (query.timeRang && query.timeRang.length === 2) {
    query.filter.gte_date_createdAt = query.timeRang[0];
    query.filter.lt_date_createdAt = query.timeRang[1];
  } else {
    ElMessage.warning("时间不能为空");
    return false;
  }
  queryLoading.value = true;
  findGoldCoinRecordList(query).then(res => {
    queryLoading.value = false;
    query.total = res.data.totalNum;
    list.value = res.data.dataList;
  }, err => {
    queryLoading.value = false;
  })
};
// 查询列表
const querySubmit = () => {
  query.pageNum = 1;
  queryList();
}
// 分页查询
const paginChange = (pageNum: any) => {
```

```

        query.pageNum = pageNum.val;
        queryList();
    };
    // 改变每页大小
    const pageSizeChange = (pageSize: any) => {
        query.pageSize = pageSize.val;
        queryList();
    };

    onMounted(() => {
        querySubmit();
    });
    return {
        shortcuts,
        timeRangChange,
        query,
        list,
        queryLoading,
        querySubmit,
        paginChange,
        pageSizeChange,
    };
},
});
</script>
<style lang="less" scoped>
.content-frame {
    height: 100%;
    min-height: 520px;
    overflow-y: auto;
    .paper-form {
        height: 50px;
        overflow-y: auto;
    }
    .bigContent {
        border: 1px solid #e0e0e0;
        .itemArr {
            height: calc(100vh - 150px - 50px);
            @media screen and (max-height: 600px) {
                height: 350px;
            }
        }
    }
}
}
:deep(.el-select) {
    width: 100%;
}
:deep(.el-dialog__header) {
    background-color: #f1f1f1;
    margin-right: 0;
}

```

```

}
:deep(.el-dialog__footer) {
  background-color: #f1f1f1;
  padding-top: 15px;
  padding-bottom: 15px;
}
</style>
<!--
@desc 金币记录
-->
<template>
  <div class="content-frame">
    <!-- 筛选 -->
    <div class="paper-form">
      <el-form :inline="true">
        <el-form-item label="用户 ID:" style="width:260px;">
          <el-input clearable v-model="query.filter.is_str_userId" placeholder="用户 ID"></el-input>
        </el-form-item>
        <el-form-item label="时间:" style="width: 430px">
          <el-date-picker v-model="query.timeRang" type="datetimerange"
value-format="YYYY-MM-DD HH:mm:ss"
start-placeholder="开始时间" end-placeholder="结束时间"
:shortcuts="shortcuts" @change="timeRangChange">
          </el-date-picker>
        </el-form-item>
        <el-form-item>
          <el-button type="primary" @click="querySubmit" :loading="queryLoading">查询</el-button>
        </el-form-item>
      </el-form>
    </div>
    <!-- 列表 -->
    <div class="bigContent">
      <div class="itemArr">
        <el-table :data="list" height="100%" v-loading="queryLoading"
stripe :header-cell-style="{ background: '#fafafa' }">
          <el-table-column label="用户 ID" prop="userId"></el-table-column>
          <el-table-column label="第几天" prop="dayOfWeek"></el-table-column>
          <el-table-column label="奖励值" prop="awardValue"></el-table-column>
          <el-table-column label="奖励类型" prop="awardType">
            <template #default="scope">
              {{getAwardTypeText(scope.row.awardType)}}
            </template>
          </el-table-column>
          <el-table-column label="是否双倍" prop="isDouble"></el-table-column>
          <el-table-column label="时间" prop="createdAt"></el-table-column>
        </el-table>
      </div>
    <!-- 分页 -->

```

```
<pagination :total="query.total" :currentPage="query.pageNum" :defaultPageSize="query.
pageSize"
    @pageChange="pageChange" @pageSizeChange="pageSizeChange"></pagination>
</div>
</div>
</template>
<script lang="ts">
import { defineComponent, onMounted, reactive, ref } from "vue";
import pagination from "@components/pagination.vue";
import { ElMessage } from "element-plus";
import { findSignInDailyRecordList } from "../../tools/log";
import { timeformat, addDays, timeSelect } from "../../tools/dateTool";
export default defineComponent({
  name: "signInRecord",
  components: {
    pagination,
  },
  setup() {
    const shortcuts = timeSelect();
    const awardTypeList = reactive<Array<any>>([
      // { value: 1, label: '去除 50%道具' },
      // { value: 2, label: '正确答案道具' },
      { value: 3, label: '复活道具' },
      { value: 10, label: '金币' },
    ]);
    const getAwardTypeText = (awardTypeCode: number) => {
      const awardType = awardTypeList.find((item) => {
        return item.value === awardTypeCode;
      });
      return awardType ? awardType.label : awardTypeCode;
    };
    const timeRangChange = (data: any) => {
      if(data == null) {
        ElMessage.warning("时间不能为空");
        return false;
      }
      const oDate1: any = new Date(data[0]).getTime();
      const oDate2: any = new Date(data[1]).getTime();
      console.log('oDate2 - oDate1', oDate2 - oDate1)
      const oneDay = 24 * 3600 * 1000;
      if (query.filter.is_str_userId && query.filter.is_str_userId.length > 8) {
        if (oDate2 - oDate1 > oneDay * 30) {
          ElMessage.warning("有用户 ID 的时间范围不能大于 30 天");
          query.timeRang[1] = timeformat(new Date(oDate1 + oneDay * 30));
          return false;
        }
      }
      else if (oDate2 - oDate1 > oneDay) {
        ElMessage.warning("没有用户 ID 的时间范围不能大于 1 天");
        query.timeRang[1] = timeformat(new Date(oDate1 + oneDay));
      }
    };
  },
});
```

```
        return false;
    }
    return true;
};
const list = ref<any[]>([]);
const queryLoading = ref<boolean>(false);
const query = reactive<any>({
    total: 0,
    pageNum: 1,
    pageSize: 20,
    sortField: "createdAt",
    sortOrder: "desc",
    timeRang: [timeformat(addDays(0)), timeformat(addDays(1))],
    filter: {},
});
// 查询列表
const queryList = () => {
    if (query.timeRang && query.timeRang.length === 2) {
        query.filter.gte_date_createdAt = query.timeRang[0];
        query.filter.lt_date_createdAt = query.timeRang[1];
    } else {
        ElMessage.warning("时间不能为空");
        return false;
    }
    queryLoading.value = true;
    findSignInDailyRecordList(query).then(res => {
        queryLoading.value = false;
        query.total = res.data.totalNum;
        list.value = res.data.dataList;
    }, err => {
        queryLoading.value = false;
    })
};
// 查询列表
const querySubmit = () => {
    query.pageNum = 1;
    queryList();
}
// 分页查询
const paginChange = (pageNum: any) => {
    query.pageNum = pageNum.val;
    queryList();
};
// 改变每页大小
const pageSizeChange = (pageSize: any) => {
    query.pageSize = pageSize.val;
    queryList();
};
```

```
    onMounted(() => {
      querySubmit();
    });
    return {
      shortcuts,
      getAwardTypeText,
      timeRangChange,
      query,
      list,
      queryLoading,
      querySubmit,
      paginChange,
      pageSizeChange,
    };
  },
});
</script>
<style lang="less" scoped>
.content-frame {
  height: 100%;
  min-height: 520px;
  overflow-y: auto;
  .paper-form {
    height: 50px;
    overflow-y: auto;
  }
  .bigContent {
    border: 1px solid #e0e0e0;
    .itemArr {
      height: calc(100vh - 150px - 50px);
      @media screen and (max-height: 600px) {
        height: 350px;
      }
    }
  }
}
:deep(.el-select) {
  width: 100%;
}
:deep(.el-dialog__header) {
  background-color: #f1f1f1;
  margin-right: 0;
}
:deep(.el-dialog__footer) {
  background-color: #f1f1f1;
  padding-top: 15px;
  padding-bottom: 15px;
}
</style>
<!--
```


@desc 答题日志

-->

<template>

<div class="content-frame">

<!-- 筛选 -->

<div class="paper-form">

<el-form :inline="true">

<el-form-item label="用户 ID:" style="width:260px;">

<el-input clearable v-model="query.filter.is_str_userId" placeholder="用户 ID"></el-input>

</el-form-item>

<el-form-item label="时间:" style="width: 430px">

<el-date-picker v-model="query.timeRang" type="datetimerange" value-format="YYYY-MM-DD HH:mm:ss" start-placeholder="开始时间" end-placeholder="结束时间" :shortcuts="shortcuts" @change="timeRangChange">

</el-date-picker>

</el-form-item>

<el-form-item>

<el-button type="primary" @click="querySubmit" :loading="queryLoading">查询</el-button>

</el-form-item>

</el-form>

</div>

<!-- 列表 -->

<div class="bigContent">

<div class="itemArr">

<el-table :data="list" height="100%" v-loading="queryLoading" stripe :header-cell-style="{ background: '#fafafa' }">

<el-table-column label="昵称" prop="nickName"></el-table-column>

<el-table-column label="用户 ID" prop="userId"></el-table-column>

<el-table-column label="题目组 ID" prop="questionGroupId"></el-table-column>

<!-- <el-table-column label="省份" prop="province"></el-table-column> -->

<el-table-column label="名次" prop="rank" width="80"></el-table-column>

<el-table-column label="对几题" prop="rightCount" width="80"></el-table-column>

<el-table-column label="是否双倍" prop="isDouble" width="80"></el-table-column>

<el-table-column label="是否保星" prop="isKeep" width="80"></el-table-column>

<el-table-column label="奖励金币" prop="goldCoin" width="80"></el-table-column>

<el-table-column label="剩余金币" prop="usableGoldCoin" width="80"></el-table-column>

<el-table-column label="段位" prop="dan" width="120">

<template #default="scope">

{{getDanText(scope.row.dan)}} {{scope.row.danStar + '星'}}

</template>

</el-table-column>

<!-- <el-table-column label="段位星" prop="danStar" width="80"></el-table-column> -->

```

        <el-table-column label="时间" prop="createdAt"></el-table-column>
      </el-table>
    </div>
    <!-- 分页 -->

    <pagination :total="query.total" :currentPage="query.pageNum" :defaultPageSize="query.
    pageSize"
      @paginChange="paginChange" @pageSizeChange="pageSizeChange"></pagination>
    </div>
  </div>
</template>
<script lang="ts">
import { defineComponent, onMounted, reactive, ref } from "vue";
import pagination from "@components/pagination.vue";
import { ElMessage } from "element-plus";
import { findUserAnswerLogList } from "../../tools/log";
import { timeformat, addDays, timeSelect } from "../../tools/dateTool";
export default defineComponent({
  name: "userAnswerLog",
  components: {
    pagination,
  },
  setup() {
    const shortcuts = timeSelect();
    const timeRangChange = (data: any) => {
      if(data == null) {
        ElMessage.warning("时间不能为空");
        return false;
      }
      const oDate1: any = new Date(data[0]).getTime();
      const oDate2: any = new Date(data[1]).getTime();
      console.log('oDate2 - oDate1', oDate2 - oDate1)
      const oneDay = 24 * 3600 * 1000;
      if (query.filter.is_str_userId && query.filter.is_str_userId.length > 8) {
        if (oDate2 - oDate1 > oneDay * 30) {
          ElMessage.warning("有用户 ID 的时间范围不能大于 30 天");
          query.timeRang[1] = timeformat(new Date(oDate1 + oneDay * 30));
          return false;
        }
      } else if (oDate2 - oDate1 > oneDay) {
        ElMessage.warning("没有用户 ID 的时间范围不能大于 1 天");
        query.timeRang[1] = timeformat(new Date(oDate1 + oneDay));
        return false;
      }
      return true;
    };

    //段位
    const danList = reactive<Array<any>>([
      { value: 1, label: '奋力黑铁' },

```

```
{ value: 2, label: '见习青铜' },
{ value: 3, label: '新晋白银' },
{ value: 4, label: '精英黄金' },
{ value: 5, label: '资深钻石' },
{ value: 6, label: '广博大师' },
{ value: 7, label: '鸿儒传奇' },
{ value: 8, label: '头脑霸主' },
]);
const getDanText = (danCode: number) => {
  const dan = danList.find((item => {
    return item.value === danCode;
  }));
  return dan ? dan.label : danCode;
};
const list = ref<any[]>([]);
const queryLoading = ref<boolean>(false);
const query = reactive<any>({
  total: 0,
  pageNum: 1,
  pageSize: 20,
  sortField: "createdAt",
  sortOrder: "desc",
  timeRang: [timeformat(addDays(0)), timeformat(addDays(1))],
  filter: {},
});
// 查询列表
const queryList = () => {
  if (query.timeRang && query.timeRang.length === 2) {
    query.filter.gte_date_createdAt = query.timeRang[0];
    query.filter.lt_date_createdAt = query.timeRang[1];
  } else {
    ElMessage.warning("时间不能为空");
    return false;
  }
  queryLoading.value = true;
  findUserAnswerLogList(query).then(res => {
    queryLoading.value = false;
    query.total = res.data.totalNum;
    list.value = res.data.dataList;
  }, err => {
    queryLoading.value = false;
  });
};
// 查询列表
const querySubmit = () => {
  query.pageNum = 1;
  queryList();
}
// 分页查询
```

```

    const paginChange = (pageNum: any) => {
      query.pageNum = pageNum.val;
      queryList();
    };
    // 改变每页大小
    const pageSizeChange = (pageSize: any) => {
      query.pageSize = pageSize.val;
      queryList();
    };
    onMounted(() => {
      querySubmit();
    });
    return {
      shortcuts,
      timeRangChange,
      danList,
      getDanText,
      query,
      list,
      queryLoading,
      querySubmit,
      paginChange,
      pageSizeChange,
    };
  },
});
</script>
<style lang="less" scoped>
.content-frame {
  height: 100%;
  min-height: 520px;
  overflow-y: auto;
  .paper-form {
    height: 50px;
    overflow-y: auto;
  }
  .bigContent {
    border: 1px solid #e0e0e0;
    .itemArr {
      height: calc(100vh - 150px - 50px);
      @media screen and (max-height: 600px) {
        height: 350px;
      }
    }
  }
}
:deep(.el-select) {
  width: 100%;
}
:deep(.el-dialog__header) {

```

```

    background-color: #f1f1f1;
    margin-right: 0;
}
:deep(.el-dialog__footer) {
    background-color: #f1f1f1;
    padding-top: 15px;
    padding-bottom: 15px;
}
</style>
<!--
@desc 排位奖励规则
-->
<template>
    <div class="content-frame">
        <!-- 列表 -->
        <div class="bigContent">
            <div class="itemArr">
                <el-table :data="list" height="100%" v-loading="queryLoading"
stripe :header-cell-style="{ background: '#fafafa' }">
                    <el-table-column label="图标" width="90">
                        <template #default="scope">
                            <el-avatar v-if="scope.row.icon" :src="scope.row.icon" :size="60"/>
                        </template>
                    </el-table-column>
                    <el-table-column label="第几天" prop="dayOfWeek"></el-table-column>
                    <el-table-column label="标题" prop="title"></el-table-column>
                    <el-table-column label="奖励数量" prop="awardValue"></el-table-column>
                    <el-table-column label="奖励类型" prop="awardType">
                        <template #default="scope">
                            {{getAwardTypeText(scope.row.awardType)}}
                        </template>
                    </el-table-column>
                    <el-table-column label="奖励说明" prop="awardDesc"></el-table-column>
                    <el-table-column label="双倍奖励说明" prop="awardDesc2"></el-table-column>
                    <el-table-column label="操作" fixed="right" width="90">
                        <template #default="scope">
                            <el-button type="primary" size="small" @click="handleEdit(scope.row)">编辑</el-button>
                        </template>
                    </el-table-column>
                </el-table>
            </div>
        </div>
    </div>

    <el-dialog :title="editTitle" v-model="showEdit"
width="760px" :close-on-click-modal="false" @close="closeEdit">
        <el-form :model="editData" label-width="120px" ref="editFormRef"
class="edit-form">
            <el-row>
                <el-col :span="12">

```

```

    <el-form-item label="标题" prop="title" :rules="[{ required: true, message:
'请输入标题' }]">
      <el-input v-model="editData.title" maxlength="16"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="第几天" prop="dayOfWeek" :rules="[{ required: true,
message: '请输入第几天' }]">
      <el-input-number
v-model="editData.dayOfWeek" :step="1" :precision="0" :min="1" :max="7"></el-input-numbe
r>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="奖励类型" prop="awardType" :rules="[{ required: true,
message: '请选择奖励了类型' }]">
      <el-select v-model="editData.awardType">
        <el-option
          v-for="item
awardTypeList" :label="item.label" :value="item.value"></el-option>
      </el-select>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="奖励数量" prop="awardValue" :rules="[{ required: true,
message: '请输入奖励数量' }]">
      <el-input-number
v-model="editData.awardValue" :step="1" :precision="0" :min="1"></el-input-number>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="奖励说明" prop="awardDesc" :rules="[{ required: true,
message: '请输入奖励说明' }]">
      <el-input v-model="editData.awardDesc" maxlength="16"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="12">
    <el-form-item label="双倍奖励说明" prop="awardDesc2" :rules="[{ required:
true, message: '请输入双倍奖励说明' }]">
      <el-input v-model="editData.awardDesc2" maxlength="16"></el-input>
    </el-form-item>
  </el-col>
  <el-col :span="24">
    <el-form-item label="图标" prop="icon" :rules="[{ required: true, message: '
请上传图片' }, {type: 'url', message: 'URL 格式不正确'}]">
      <div style="width: 100%; height: 255px;">
        <div style="width: 100%; height: 35px;">
          <el-input v-model="editData.icon" placeholder="还可直接输入图片
URL"></el-input>
        </div>
      <div style="width: 100%; height: 200px; display: flex; flex-direction: row;

```

```

justify-content: space-between;">
      <el-upload      drag      :auto-upload="false"      :show-file-list="false"
accept="image/png, image/jpeg" :on-change="uploadImg"
      style="width: 80%; height: 100%;">
        <template #default>
          <el-icon class="el-icon--upload"> <upload-filled /> </el-icon>
          <div class="el-upload__text"> 拖拽到此区域 | <em>点击选择图片
</em></div>

          </template>
          <template #tip>
            <div>请上传 1 张 jpg/png 图片，大小不超过 50K</div>
          </template>
        </el-upload>
        <el-image
v-if="editData.icon"      :src="editData.icon"      :preview-src-list="[editData.icon]"
fit="contain"
      style="width: 15%; height: 100%;"></el-image>
    </div>
  </div>
</el-form-item>
</el-col>
</el-row>
</el-form>
<template #footer>
  <div class="dialog-footer">
    <el-button @click="closeEdit">取 消</el-button>
    <el-button type="primary" @click="submitEdit">提 交</el-button>
  </div>
</template>
</el-dialog>
</div>
</template>
<script lang="ts">
import { defineComponent, onMounted, ref, reactive } from "vue";
import pagination from "@components/pagination.vue";
import { ElMessage } from "element-plus";
import { findAllSignInSettings, saveSignInSettings } from "../../tools/sys";
import { findAliOssToken, uploadFileToAliOss } from "../../tools/uploadOss";
export default defineComponent({
  name: "signInSettings",
  components: {
    pagination,
  },
  setup() {
    const awardTypeList = reactive<Array<any>>([
      { value: 1, label: '去除 50%道具' },
      { value: 2, label: '正确答案道具' },
      { value: 3, label: '复活道具' },
      { value: 10, label: '金币' },
    ]);
  }
});

```

```
const getAwardTypeText = (awardTypeCode: number) => {
  const awardType = awardTypeList.find((item => {
    return item.value === awardTypeCode;
  }));
  return awardType ? awardType.label : awardTypeCode;
};
const list = ref<any[]>([]);
const queryLoading = ref<boolean>(false);
// 查询列表
const queryList = () => {
  queryLoading.value = true;
  findAllSignInSettings().then(res => {
    queryLoading.value = false;
    list.value = res.data.dataList;
  }, err => {
    queryLoading.value = false;
  })
};
const showEdit = ref<boolean>(false);
const editData = ref<any>({});
const editTitle = ref<string>('');
const editFormRef = ref<any>({});
//新增
const handleAdd = () => {
  const nextTop = (list.value && list.value.length) ? list.value[list.value.length - 1].end + 1 : 1;
  editData.value = {
    id: undefined,
    start: nextTop,
    end: nextTop + 1,
  };
  editTitle.value = '新增';
  showEdit.value = true;
};
//编辑
const handleEdit = (item: any) => {
  editData.value = JSON.parse(JSON.stringify(item));
  editTitle.value = '编辑';
  showEdit.value = true;
};
//关闭
const closeEdit = () => {
  showEdit.value = false;
}
//提交
const submitEdit = () => {
  editFormRef.value!.validate((isOk: boolean) => {
    if (isOk) {
      saveSignInSettings(editData.value).then((res: any) => {
```



```
        showEdit.value = false;
        editFormRef.value.resetFields();
        queryList();
    });
    } else {
        ElMessage.warning("请正确填写完整信息");
    }
});
}
//上传图片
const uploadImg = (uploadFile: any) => {
    const file = uploadFile.raw;
    const fileSize = uploadFile.size / 1024;
    if (fileSize > 60) {
        ElMessage.warning("图片大小不能超过 50K");
        return false;
    }
    //获取阿里云配置并上传
    findAliOssToken().then((ossConfig) => {
        uploadFileToAliOss(file, "sys", ref<any>(ossConfig)).then((res) => {
            editData.value.icon
        });
    });
}
onMounted(() => {
    queryList();
});
return {
    awardTypeList,
    getAwardTypeText,
    list,
    queryLoading,
    showEdit,
    editData,
    editTitle,
    editFormRef,
    handleAdd,
    handleEdit,
    submitEdit,
    closeEdit,
    uploadImg,
};
},
});
</script>
<style lang="less" scoped>
.content-frame {
    height: 100%;
    min-height: 520px;
    overflow-y: auto;
```

```

    .paper-form {
      height: 50px;
      overflow-y: auto;
    }
    .bigContent {
      border: 1px solid #e0e0e0;
      .itemArr {
        height: calc(100vh - 100px);
        @media screen and (max-height: 600px) {
          height: 350px;
        }
      }
    }
  }
}
:deep(.el-select) {
  width: 100%;
}
:deep(.el-dialog__header) {
  background-color: #f1f1f1;
  margin-right: 0;
}
:deep(.el-dialog__footer) {
  background-color: #f1f1f1;
  padding-top: 15px;
  padding-bottom: 15px;
}
</style>
<!--
@desc 排位奖励规则
-->
<template>
  <div class="content-frame">
    <!-- 列表 -->
    <div class="bigContent">
      <div class="itemArr">
        <el-table :data="list" height="100%" v-loading="queryLoading"
        stripe :header-cell-style="{ background: '#fafafa' }">
          <el-table-column label="图标" width="90">
            <template #default="scope">
              <el-avatar v-if="scope.row.icon" :src="scope.row.icon" :size="60"/>
            </template>
          </el-table-column>
          <el-table-column label="任务类型" prop="taskType">
            <template #default="scope">
              {{getTaskTypeText(scope.row.taskType)}}
            </template>
          </el-table-column>
          <el-table-column label="标题" prop="title"></el-table-column>
          <el-table-column label="副标题" prop="awardDesc"></el-table-column>
          <el-table-column label="需完成次数" prop="needCount"></el-table-column>

```

```

    <el-table-column label="奖品图标" width="90">
      <template #default="scope">
        <el-avatar
v-if="scope.row.awardIcon" :src="scope.row.awardIcon" :size="60"/>
      </template>
    </el-table-column>
    <el-table-column label="奖品数量" prop="awardValue"></el-table-column>
    <el-table-column label="奖品类型" prop="awardType">
      <template #default="scope">
        {{getAwardTypeText(scope.row.awardType)}}
      </template>
    </el-table-column>
    <el-table-column label="奖品说明" prop="awardDesc2"></el-table-column>
    <el-table-column label="操作" fixed="right" width="90">
      <template #default="scope">
        <el-button type="primary" size="small" @click="handleEdit(scope.row)">编辑</el-button>
      </template>
    </el-table-column>
  </el-table>
</div>
</div>

<el-dialog :title="editTitle" v-model="showEdit"
width="760px" :close-on-click-modal="false" @close="closeEdit">
  <el-form :model="editData" label-width="120px" ref="editFormRef"
class="edit-form">
    <el-row>
      <el-col :span="12">
        <el-form-item label="任务类型" prop="taskType" :rules="[{ required: true,
message: '请选择任务类型' }]">
          <el-select v-model="editData.taskType" :disabled="editData.taskType">
            <el-option v-for="item in taskTypeList" :label="item.label" :value="item.value"></el-option>
          </el-select>
        </el-form-item>
      </el-col>
      <el-col :span="12">
        <el-form-item label="序号(正序)" prop="orderNum" :rules="[{ required: true,
message: '请输入序号' }]">
          <el-input-number
v-model="editData.orderNum" :step="1" :precision="0" :min="1"></el-input-number>
        </el-form-item>
      </el-col>
      <el-col :span="12">
        <el-form-item label="标题" prop="title" :rules="[{ required: true, message:
'请输入标题' }]">
          <el-input v-model="editData.title" maxlength="32"></el-input>
        </el-form-item>
      </el-col>
    </el-row>
  </el-form>
</div>

```

```

        <el-col :span="12">
            <el-form-item label="副标题" prop="awardDesc" :rules="[{ required: true,
message: '请输入副标题' }]">
                <el-input v-model="editData.awardDesc" maxlength="32"></el-input>
            </el-form-item>
        </el-col>
        <el-col :span="12">
            <el-form-item label="奖品类型" prop="awardType" :rules="[{ required: true,
message: '请选择奖品类型' }]">
                <el-select v-model="editData.awardType">
                    <el-option v-for="item in
awardTypeList" :label="item.label" :value="item.value"></el-option>
                </el-select>
            </el-form-item>
        </el-col>
        <el-col :span="12">
            <el-form-item label="奖品数量" prop="awardValue" :rules="[{ required: true,
message: '请输入奖品数量' }]">
                <el-input-number
v-model="editData.awardValue" :step="1" :precision="0" :min="1"></el-input-number>
            </el-form-item>
        </el-col>
        <el-col :span="12">
            <el-form-item label="奖品说明" prop="awardDesc2" :rules="[{ required: true,
message: '请输入奖品说明' }]">
                <el-input v-model="editData.awardDesc2" maxlength="16"></el-input>
            </el-form-item>
        </el-col>
        <el-col :span="12">
            <el-form-item label="需完成次数" prop="needCount" :rules="[{ required: true,
message: '请输入需完成次数' }]">
                <el-input-number
v-model="editData.needCount" :step="1" :precision="0" :min="1" :disabled="editData.taskT
ype != 3"></el-input-number>
            </el-form-item>
        </el-col>
        <el-col :span="24">
            <el-form-item label="图标" prop="icon" :rules="[{ required: true, message: '
请上传图片' }, {type: 'url', message: 'URL 格式不正确'}]">
                <div style="width: 100%; height: 255px;">
                    <div style="width: 100%; height: 35px;">
                        <el-input v-model="editData.icon" placeholder="还可直接输入图片
URL"></el-input>
                    </div>
                <div style="width: 100%; height: 200px; display: flex; flex-direction: row;
justify-content: space-between;">
                    <el-upload drag :auto-upload="false" :show-file-list="false"
accept="image/png, image/jpeg" :on-change="uploadImg"
style="width: 80%; height: 100%;">
                        <template #default>

```

```

        <el-icon class="el-icon--upload"> <upload-filled /> </el-icon>
        <div class="el-upload__text"> 拖拽到此区域 | <em>点击选择图片
</em></div>

    </template>
    <template #tip>
        <div>请上传 1 张 jpg/png 图片，大小不超过 50K</div>
    </template>
</el-upload>
<el-image
v-if="editData.icon"      :src="editData.icon"      :preview-src-list="[editData.icon]"
fit="contain"
    style="width: 15%; height: 100%;"></el-image>
</div>
</div>
</el-form-item>
</el-col>
<el-col :span="24">
    <el-form-item label="奖品图标" prop="awardIcon" :rules="[{ required: true,
message: '请上传图片' }, {type: 'url', message: 'URL 格式不正确'}]">
        <div style="width: 100%; height: 255px;">
            <div style="width: 100%; height: 35px;">
                <el-input v-model="editData.awardIcon" placeholder="还可直接输入图片
URL"></el-input>
            </div>
            <div style="width: 100%; height: 200px; display: flex; flex-direction: row;
justify-content: space-between;">
                <el-upload drag :auto-upload="false" :show-file-list="false"
accept="image/png, image/jpeg" :on-change="uploadImg2"
                    style="width: 80%; height: 100%;">
                    <template #default>
                        <el-icon class="el-icon--upload"> <upload-filled /> </el-icon>
                        <div class="el-upload__text"> 拖拽到此区域 | <em>点击选择图片
</em></div>
                    </template>
                    <template #tip>
                        <div>请上传 1 张 jpg/png 图片，大小不超过 50K</div>
                    </template>
                </el-upload>
                <el-image
v-if="editData.awardIcon" :src="editData.awardIcon" :preview-src-list="[editData.awardI
con]" fit="contain"
                    style="width: 15%; height: 100%;"></el-image>
                </div>
            </div>
        </el-form-item>
    </el-col>
</el-row>
</el-form>
<template #footer>
    <div class="dialog-footer">

```

```
<el-button @click="closeEdit">取 消</el-button>
<el-button type="primary" @click="submitEdit">提 交</el-button>
</div>
</template>
</el-dialog>
</div>
</template>
<script lang="ts">
import { defineComponent, onMounted, ref, reactive } from "vue";
import pagination from "@components/pagination.vue";
import { ElMessage } from "element-plus";
import { findAllTaskBase, saveTaskBase } from "../../tools/sys";
import { findAliOssToken, uploadFileToAliOss } from "../../tools/uploadOss";
export default defineComponent({
  name: "taskBase",
  components: {
    pagination,
  },
  setup() {
    const awardTypeList = reactive<Array<any>>([
      { value: 1, label: '去除 50%道具' },
      { value: 2, label: '正确答案道具' },
      { value: 3, label: '复活道具' },
      { value: 10, label: '金币' },
    ]);
    const getAwardTypeText = (awardTypeCode: number) => {
      const awardType = awardTypeList.find((item => {
        return item.value === awardTypeCode;
      }));
      return awardType ? awardType.label : awardTypeCode;
    };
    const taskTypeList = reactive<Array<any>>([
      { value: 1, label: '每日参与排位赛' },
      { value: 2, label: '每日观看激励广告' },
      { value: 3, label: '每日累计观看激励广告' },
      { value: 4, label: '每日成功吃鸡' },
    ]);
    const getTaskTypeText = (taskTypeCode: number) => {
      const taskType = taskTypeList.find((item => {
        return item.value === taskTypeCode;
      }));
      return taskType ? taskType.label : taskTypeCode;
    };
    const list = ref<any[]>([]);
    const queryLoading = ref<boolean>(false);
    // 查询列表
    const queryList = () => {
      queryLoading.value = true;
      findAllTaskBase().then(res => {
        queryLoading.value = false;
      });
    };
  },
});
```

```
        list.value = res.data.dataList;
      }, err => {
        queryLoading.value = false;
      }
    );
  };
  const showEdit = ref<boolean>(false);
  const editData = ref<any>({});
  const editTitle = ref<string>('');
  const editFormRef = ref<any>({});
  //新增
  const handleAdd = () => {
    const nextTop = (list.value && list.value.length) ? list.value[list.value.length -
1].end + 1 : 1;
    editData.value = {
      id: undefined,
      start: nextTop,
      end: nextTop + 1,
    };
    editTitle.value = '新增';
    showEdit.value = true;
  };
  //编辑
  const handleEdit = (item: any) => {
    editData.value = JSON.parse(JSON.stringify(item));
    editTitle.value = '编辑';
    showEdit.value = true;
  };
  //关闭
  const closeEdit = () => {
    showEdit.value = false;
  }
  //提交
  const submitEdit = () => {
    editFormRef.value!.validate((isOk: boolean) => {
      if (isOk) {
        saveTaskBase(editData.value).then((res: any) => {
          showEdit.value = false;
          editFormRef.value.resetFields();
          queryList();
        });
      } else {
        ElMessage.warning("请正确填写完整信息");
      }
    });
  };
  //上传图片
  const uploadImg = (uploadFile: any) => {
    const file = uploadFile.raw;
    const fileSize = uploadFile.size / 1024;
```

```
        if (fileSize > 60) {
            ElMessage.warning("图片大小不能超过 50K");
            return false;
        }
        //获取阿里云配置并上传
        findAliOssToken().then((ossConfig) => {
            uploadFileToAliOss(file, "sys", ref<any>(ossConfig)).then((res) => {
                editData.value.icon
            });
        });
    }
    //上传图片
    const uploadImg2 = (uploadFile: any) => {
        const file = uploadFile.raw;
        const fileSize = uploadFile.size / 1024;
        if (fileSize > 60) {
            ElMessage.warning("图片大小不能超过 50K");
            return false;
        }
        //获取阿里云配置并上传
        findAliOssToken().then((ossConfig) => {
            uploadFileToAliOss(file, "sys", ref<any>(ossConfig)).then((res) => {
                editData.value.awardIcon
            });
        });
    }
    onMounted(() => {
        queryList();
    });
    return {
        awardTypeList,
        getAwardTypeText,
        taskTypeList,
        getTaskTypeText,
        list,
        queryLoading,
        showEdit,
        editData,
        editTitle,
        editFormRef,
        handleAdd,
        handleEdit,
        submitEdit,
        closeEdit,
        uploadImg,
        uploadImg2,
    };
},
});
</script>
```



```
// 查询列表
const queryList = () => {
  // if (query.timeRang && query.timeRang.length === 2) {
  //   query.filter.gte_date_createdAt = query.timeRang[0];
  //   query.filter.lt_date_createdAt = query.timeRang[1];
  // } else if(!query.filter.is_str_id){
  //   ElMessage.warning("时间和 ID 不能都为空");
  //   return false;
  // }
  queryLoading.value = true;
  findUserStatisList(query).then(res => {
    queryLoading.value = false;
    query.total = res.data.totalNum;
    list.value = res.data.dataList;
  }, err => {
    queryLoading.value = false;
  })
};

// 查询列表
const querySubmit = () => {
  query.pageNum = 1;
  queryList();
}

// 分页查询
const paginChange = (pageNum: any) => {
  query.pageNum = pageNum.val;
  queryList();
};

// 改变每页大小
const pageSizeChange = (pageSize: any) => {
  query.pageSize = pageSize.val;
  queryList();
};

onMounted(() => {
  querySubmit();
});

return {
  // shortcuts,
  danList,
  getDanText,
  query,
  list,
  queryLoading,
  querySubmit,
  paginChange,
  pageSizeChange,
};
},
});
```

```

</script>
<style lang="less" scoped>
.content-frame {
    height: 100%;
    min-height: 520px;
    overflow-y: auto;
    .paper-form {
        height: 50px;
        overflow-y: auto;
    }
    .bigContent {
        border: 1px solid #e0e0e0;
        .itemArr {
            height: calc(100vh - 150px);
            @media screen and (max-height: 600px) {
                height: 350px;
            }
        }
    }
}
:deep(.el-select) {
    width: 100%;
}
:deep(.el-dialog__header) {
    background-color: #f1f1f1;
    margin-right: 0;
}
:deep(.el-dialog__footer) {
    background-color: #f1f1f1;
    padding-top: 15px;
    padding-bottom: 15px;
}
</style>

```

```

package com.olal00.tnzb.data.domain.form;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Field;
import java.io.Serializable;
/**
@Data
@Accessors(chain = true)
public class QuestionAnswer implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    @Field("question_id")
    private String questionId; //题目分组 id
    @Field("is_super")
    private Integer isSuper; //是否终极一题
    @Field("answer")

```

```

    private String answer; //答案 ABCD, 0=A, 1=B, 2=C, 3=D
    @Field("answer_index")
    private Integer answerIndex; //答案从 0 开始, 0=A, 1=B, 2=C, 3=D
    @Field("checked_answer")
    private String checkedAnswer; //答案 ABCD, 0=A, 1=B, 2=C, 3=D
    @Field("checked_answer_index")
    private Integer checkedAnswerIndex; //答案从 0 开始, 0=A, 1=B, 2=C, 3=D
    @Field("is_right")
    private Integer isRight; //是否回答正确
    @Field("answer_second")
    private Integer answerSecond; //答题时间
    @Field("is_use_prop")
    private Integer isUseProp; //是否使用了道具
    @Field("is_relive")
    private Integer isRelive; //是否使用了复活
}

package com.olal00.tnzb.data.domain.form;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import org.springframework.data.mongodb.core.mapping.Field;
import java.io.Serializable;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
/**
@Data
@Accessors(chain = true)
public class TaskState implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    @Field("open_qualifying_time")
    private LocalDateTime openQualifyingTime; //最后一次参与排位赛时间
    @Field("open_qualifying_award_time")
    private LocalDateTime openQualifyingAwardTime; //最后一次参与排位赛领奖时间
    @Field("win_qualifying_time")
    private LocalDateTime winQualifyingTime; //最后一次排位赛吃鸡时间
    @Field("win_qualifying_award_time")
    private LocalDateTime winQualifyingAwardTime; //最后一次排位赛吃鸡领奖时间
    @Field("view_ad_time")
    private LocalDateTime viewAdTime; //最后一次观看激励广告(任务)时间
    @Field("view_ad_award_time")
    private LocalDateTime viewAdArardTime; //最后一次观看激励广告(任务)领奖时间
    @Field("view_ads_award_time")
    private LocalDateTime viewAdsArardTime; //最后一次单日累计观看激励广告(任务)领奖时间
    @Field("today_view_ad_count")
    private Integer todayViewAdCount; //今日观看激励广告(任务)次数
    public Integer getTodayViewAdCount() {
        if(this.getViewAdTime() == null || todayViewAdCount == null || todayViewAdCount
< 1) {
            return 0;

```

```
        }
        if (this.getViewAdTime().isBefore(LocalDateTime.of(LocalDate.now(),
LocalTime.MIN))) {
            return 0;
        }
        return todayViewAdCount;
    }
}

package com.olal00.tnzb.admin.log.web;
import cn.dev33.satoken.annotation.SaCheckPermission;
import cn.dev33.satoken.annotation.SaMode;
import com.olal00.tnzb.admin.form.req ReqObj;
import com.olal00.tnzb.admin.log.service.SignInDailyRecordService;
import com.olal00.tnzb.common.config.VlRestController;
import com.olal00.tnzb.common.response.PageRes;
import com.olal00.tnzb.common.response.Ret;
import com.olal00.tnzb.common.response.RetUtil;
import com.olal00.tnzb.data.domain.common.SignInDailyRecord;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.ResponseBody;
/**
@VlRestController("/signInDaily")
public class SignInDailyRecordController {
    @Autowired
    private SignInDailyRecordService signInDailyRecordService;
    //分页查询
    @SaCheckPermission(value = {"signInDaily-list", "all"}, mode = SaMode.OR)
    @PostMapping("/list")
    public Ret<PageRes<SignInDailyRecord>> list(@RequestBody ReqObj reqObj) throws
Exception {
        return RetUtil.ok(signInDailyRecordService.findPage(reqObj));
    }
}

package com.olal00.tnzb.admin.sys.web;
import cn.dev33.satoken.annotation.SaCheckPermission;
import cn.dev33.satoken.annotation.SaMode;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.admin.form.req ReqObj;
import com.olal00.tnzb.admin.sys.service.SignInSettingsService;
import com.olal00.tnzb.common.config.VlRestController;
import com.olal00.tnzb.common.enums.EnumSignAwardType;
import com.olal00.tnzb.common.response.ListRes;
import com.olal00.tnzb.common.response.Ret;
import com.olal00.tnzb.common.response.RetUtil;
import com.olal00.tnzb.common.util.IdUtil;
import com.olal00.tnzb.data.domain.common.SignInSettings;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
```

```

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import java.time.LocalDateTime;
/**
@V1RestController("/signInSettings")
public class SignInController {
    @Autowired
    private SignInSettingsService signInSettingsService;
    //查询全部
    @SaCheckPermission(value = {"signInSettings-findAll", "all"}, mode = SaMode.OR)
    @GetMapping("/findAll")
    public Ret<ListRes<SignInSettings>> findAll() throws Exception {
        return
RetUtil.ok(ListRes.of(signInSettingsService.findAll(Sort.by(Sort.Order.asc("dayOfWeek")
)))));
    }
    //编辑保存
    @SaCheckPermission(value = {"signInSettings-save", "all"}, mode = SaMode.OR)
    @PostMapping("/save")
    public Ret<Object> save(@RequestBody SignInSettings signInSettings) throws Exception
{
        if(StrUtil.isBlank(signInSettings.getTitle())) {
            return RetUtil.fail("标题不能为空");
        }
        if (EnumSignAwardType.getByCode(signInSettings.getAwardType()) == null ||
signInSettings.getAwardValue() == null) {
            return RetUtil.fail("奖励不能为空");
        }
        if(StrUtil.isBlank(signInSettings.getAwardDesc()) ||
StrUtil.isBlank(signInSettings.getAwardDesc2())) {
            return RetUtil.fail("奖励说明不能为空");
        }
        if (signInSettings.getDayOfWeek() == null) {
            return RetUtil.fail("周几不能为空");
        }
        if(StrUtil.isBlank(signInSettings.getIcon())) {
            return RetUtil.fail("图标不能为空");
        }
        if (StrUtil.isBlank(signInSettings.getId())) {
            signInSettings.setId(IdUtil.nextId());
            signInSettings.setCreatedAt(LocalDateTime.now());
            signInSettingsService.insert(signInSettings);
        } else {
            signInSettingsService.update(signInSettings);
        }
        return RetUtil.ok();
    }
    //删除
    @SaCheckPermission(value = {"signInSettings-delete", "all"}, mode = SaMode.OR)

```

```
@PostMapping("/delete")
public Ret<String> delete(@RequestBody ReqObj obj) throws Exception {
    if (StringUtil.isBlank(obj.getId())) {
        return RetUtil.fail("ID 不能为空");
    }
    signInSettingsService.delete(obj.getId(), true);
    return RetUtil.ok("删除成功");
}

}

package com.olal00.tnzb.admin.sys.web;
import cn.dev33.satoken.annotation.SaCheckPermission;
import cn.dev33.satoken.annotation.SaMode;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.admin.form.req.ReqObj;
import com.olal00.tnzb.admin.sys.service.TaskBaseService;
import com.olal00.tnzb.common.config.VlRestController;
import com.olal00.tnzb.common.enums.EnumSignAwardType;
import com.olal00.tnzb.common.enums.EnumTaskType;
import com.olal00.tnzb.common.response.ListRes;
import com.olal00.tnzb.common.response.Ret;
import com.olal00.tnzb.common.response.RetUtil;
import com.olal00.tnzb.common.util.IdUtil;
import com.olal00.tnzb.data.domain.common.TaskBase;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.time.LocalDateTime;
/**
 * @VlRestController("/taskBase")
 * public class TaskBaseController {
 *     @Autowired
 *     private TaskBaseService taskBaseService;
 *     //查询全部
 *     @SaCheckPermission(value = {"taskBase-findAll", "all"}, mode = SaMode.OR)
 *     @GetMapping("/findAll")
 *     public Ret<ListRes<TaskBase>> findAll() throws Exception {
 *         return
 * RetUtil.ok(ListRes.of(taskBaseService.findAll(Sort.by(Sort.Order.asc("orderNum")))));
 *     }
 *     //编辑保存
 *     @SaCheckPermission(value = {"taskBase-save", "all"}, mode = SaMode.OR)
 *     @PostMapping("/save")
 *     public Ret<Object> save(@RequestBody TaskBase taskBase) throws Exception {
 *         if (EnumTaskType.getByCode(taskBase.getTaskType()) == null) {
 *             return RetUtil.fail("任务类型不能为空");
 *         }
 *         if(StrUtil.isBlank(taskBase.getTitle())) {
```

```

        return RetUtil.fail("标题不能为空");
    }
    if (EnumSignAwardType.getByCode(taskBase.getAwardType()) == null ||
taskBase.getAwardValue() == null) {
        return RetUtil.fail("奖励不能为空");
    }
    if(StrUtil.isBlank(taskBase.getAwardDesc())) {
        return RetUtil.fail("副标题不能为空");
    }
    if(StrUtil.isBlank(taskBase.getAwardDesc2())) {
        return RetUtil.fail("奖励说明不能为空");
    }
    if (taskBase.getOrderNum() == null) {
        return RetUtil.fail("序号不能为空");
    }
    if(StrUtil.isBlank(taskBase.getIcon())) {
        return RetUtil.fail("图标不能为空");
    }
    if(StrUtil.isBlank(taskBase.getAwardIcon())) {
        return RetUtil.fail("奖品图标不能为空");
    }
    if (taskBase.getNeedCount() == null || taskBase.getNeedCount() < 1) {
        taskBase.setNeedCount(1);
    }
    if (StrUtil.isBlank(taskBase.getId())) {
        taskBase.setId(IdUtil.nextId());
        taskBase.setCreatedAt(LocalDateTime.now());
        taskBaseService.insert(taskBase);
    } else {
        taskBaseService.update(taskBase);
    }
    return RetUtil.ok();
}
//删除
@SaCheckPermission(value = {"taskBase-delete", "all"}, mode = SaMode.OR)
@PostMapping("/delete")
public Ret<String> delete(@RequestBody ReqObj obj) throws Exception {
    if (StringUtils.isBlank(obj.getId())) {
        return RetUtil.fail("ID 不能为空");
    }
    taskBaseService.delete(obj.getId(), true);
    return RetUtil.ok("删除成功");
}
}

package com.olal00.tnzb.app.user.web;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.app.user.form.req.UserReq;
import com.olal00.tnzb.app.user.form.vo.PowerVo;
import com.olal00.tnzb.app.user.service.UserService;
import com.olal00.tnzb.app.user.service.UserStatisService;

```

```

import com.olal00.tnzb.common.TnzbConst;
import com.olal00.tnzb.common.config.VlRestController;
import com.olal00.tnzb.common.redis.RedisKey;
import com.olal00.tnzb.common.redis.RedisValue;
import com.olal00.tnzb.common.response.Ret;
import com.olal00.tnzb.common.response.RetUtil;
import com.olal00.tnzb.data.domain.common.User;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;
import java.time.Duration;

/**
 * @VlRestController("/power")
 */
public class PowerController {
    @Autowired
    private RedisValue<String> redisString;
    @Autowired
    private UserStatisService userStatisService;
    @Autowired
    private UserService userService;
    //获取精力信息
    @GetMapping("/info")
    public Ret<PowerVo> info(UserReq userReq) throws Exception {
        return RetUtil.ok(userStatisService.findPower(userReq.getUserId()));
    }
    //看广告加精力（好友助力会自动加的，不需要手动调用）
    @PostMapping("/addPower")
    public Ret<PowerVo> addPower(@RequestBody UserReq userReq) throws Exception {
        if(StrUtil.isNotBlank(userReq.getInviterId())) {
            User user = userService.findById(userReq.getInviterId());
            if(user != null && StrUtil.isNotBlank(user.getId())) {
                //1 个好友 24 小时只能助力 1 次
                String key = RedisKey.ZHU_LI + user.getId() + "_" + userReq.getUserId();
                if(redisString.hasKey(key)) {
                    //return RetUtil.fail("1 天只能助力 1 次哦");
                    return
                }
                RetUtil.ok(userStatisService.findPower(userReq.getUserId()));
                PowerVo vo = userStatisService.addPower(user.getId(),
                TnzbConst.POWER_AD_ADD);
                redisString.setValue(key, "1", Duration.ofDays(1)); //1 个好友 1 天只能
                助力 1 次
                return RetUtil.ok(vo);
            } else {
                return RetUtil.fail("邀请人不存在");
            }
        } else {
            return RetUtil.ok(userStatisService.addPower(userReq.getUserId(),
            TnzbConst.POWER_AD_ADD));
        }
    }
}

```



```

    }
}
//手动减精力（闯关时会自动减的，不需要手动调用）
@PostMapping("/usePower")
public Ret<PowerVo> usePower(@RequestBody UserReq userReq) throws Exception {
    return RetUtil.ok(userStatisService.usePower(userReq.getUserId(),
TnzbConst.POWER_USE));
}
}

package com.olal00.tnzb.app.task.form.vo;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import java.io.Serializable;
/**
@Data
@Accessors(chain = true)
public class SignInResVo implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    private Integer isSuccess; //是否成功
    private String awardDesc; //奖励
    private String icon; //icon
    private Integer usableGoldCoin; //用户可用金币数
    private Integer usableHalfProp; //50%锦囊道具数
    private Integer usableRightProp; //正确答案道具数
    private Integer usableReliveProp; //复活道具数
}

package com.olal00.tnzb.app.task.form.vo;
import cn.hutool.core.collection.ListUtil;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import java.io.Serializable;
import java.util.List;
/**
@Data
@Accessors(chain = true)
public class SignInVo implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    private List<String> tplIds = ListUtil.of(TnzbConst.SIGNIN_TMPLID); //订阅的消息模
板 id 的集合
    private List<SignIn> signIns; //签到状态
    private Integer signInCount; //本周签到次数
    private String nextAwardDesc; //明日签到可得奖励
@Data
@Accessors(chain = true)
public static class SignIn {
    private String title; //标题
    private String icon; //icon
    private String awardDesc; //奖励

```

```
        private Integer status; //1:待领取, 2:已领取, 3:已过期, 4:未开始
        private String statusText;
    }
}

package com.olal00.tnzb.app.task.form.vo;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import java.io.Serializable;
/**
@Data
@Accessors(chain = true)
public class TaskResVo implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    private Integer isSuccess; //是否成功
    private Integer status; //1:做任务, 2:待领取, 3:已发放
    private String statusText;
    private String awardDesc; //奖励
    private String icon; //icon
    private Integer usableGoldCoin; //用户可用金币数
    private Integer usableHalfProp; //50%锦囊道具数
    private Integer usableRightProp; //正确答案道具数
    private Integer usableReliveProp; //复活道具数
}

package com.olal00.tnzb.app.task.form.vo;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import java.io.Serializable;
/**
@Data
@Accessors(chain = true)
public class TaskVo implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    private String taskId; //任务 ID
    private Integer taskType; //任务类型 EnumTaskType
    private String icon; //icon
    private String title; //标题
    private String awardDesc; //奖励
    private Integer status; //1:做任务, 2:待领取, 3:已发放
    private String statusText;
}

package com.olal00.tnzb.app.task.service;
import cn.hutool.core.util.ArrayUtil;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.app.task.form.vo.SignInResVo;
import com.olal00.tnzb.app.task.form.vo.SignInVo;
import com.olal00.tnzb.common.TnzbConst;
import com.olal00.tnzb.common.config.MongoTransactional;
```

```

import com.olal00.tnzb.common.enums.*;
import com.olal00.tnzb.common.redis.RedisKey;
import com.olal00.tnzb.common.redis.RedisLock;
import com.olal00.tnzb.common.util.IdUtil;
import com.olal00.tnzb.common.util.date.DateUtil;
import com.olal00.tnzb.data.dao.MongoUtil;
import com.olal00.tnzb.data.dao.base.MongoCommonDao;
import com.olal00.tnzb.data.dao.common.GoldCoinRecordDao;
import com.olal00.tnzb.data.dao.common.PropRecordDao;
import com.olal00.tnzb.data.dao.common.UserDao;
import com.olal00.tnzb.data.dao.common.UserStatisDao;
import com.olal00.tnzb.data.domain.common.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Update;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
/**
@Service
public class SignInSettingsServiceImpl extends BaseServiceImpl<SignInSettings, String>
implements SignInSettingsService {
    @Autowired
    private RedisLock redisLock;
    @Autowired
    private UserDao userDao;
    @Autowired
    private UserStatisDao userStatisDao;
    @Autowired
    private GoldCoinRecordDao goldCoinRecordDao;
    @Autowired
    private PropRecordDao propRecordDao;
    @Autowired
    private MongoCommonDao<SignInDailyRecord, String> signInDailyRecordDao;
    @Override
    public SignInVo findSignIns(String userId) throws Exception {
        List<SignInSettings> signInSettingsList =
mongoCommonDao.findAll(Sort.by(Sort.Order.asc("dayOfWeek")),
        SignInSettings.class);
        List<SignInVo.SignIn> signInList = new ArrayList<>(signInSettingsList.size());
        LocalDate today = LocalDate.now();
        int dayOfWeek = today.getDayOfWeek().getValue();
        int week = DateUtil.getYearWeek(today);
        List<SignInDailyRecord> signInDailyRecordList =
signInDailyRecordDao.find(MongoUtil.getQuery().addCriteria(
        Criteria.where("userId").is(userId).and("week").is(week)),
SignInDailyRecord.class);

```

```

        int signInCount = 0; //本周签到次数
        String nextAwardDesc = null; //明日签到可得奖励
        for (SignInSettings signInSettings : signInSettingsList) {
            SignInVo.SignIn signIn = new SignInVo.SignIn();
            signIn.setTitle(signInSettings.getTitle());
            signIn.setIcon(signInSettings.getIcon());
            signIn.setAwardDesc(signInSettings.getAwardDesc());
            if(signInSettings.getDayOfWeek() > dayOfWeek) { //未开始
                signIn.setStatus(EnumSignInStatus.STATUS_WKS_4.getCode());
                signIn.setStatusText(EnumSignInStatus.STATUS_WKS_4.getText());
            } else {
                SignInDailyRecord signInDailyRecord =
                getSignInDailyRecord(signInDailyRecordList, signInSettings.getDayOfWeek());
                if(signInDailyRecord == null) {
                    if(signInSettings.getDayOfWeek() < dayOfWeek) { //已过期
                        signIn.setStatus(EnumSignInStatus.STATUS_YGQ_3.getCode());
                        signIn.setStatusText(EnumSignInStatus.STATUS_YGQ_3.getText());
                    } else { //待领取
                        signIn.setStatus(EnumSignInStatus.STATUS_DLQ_1.getCode());
                        signIn.setStatusText(EnumSignInStatus.STATUS_DLQ_1.getText());
                    }
                } else { //已领取
                    signIn.setStatus(EnumSignInStatus.STATUS_YLQ_2.getCode());
                    signIn.setStatusText(EnumSignInStatus.STATUS_YLQ_2.getText());
                    signInCount++;
                }
            }
            if((dayOfWeek == 7 && signInSettings.getDayOfWeek() == 1) || (dayOfWeek + 1
            == signInSettings.getDayOfWeek())) {
                nextAwardDesc = signInSettings.getAwardDesc();
            }
            signInList.add(signIn);
        }
        return new
        SignInVo().setSignInList(signInList).setSignInCount(signInCount).setNextAwardDesc(nextAwardDesc);
    }
    @MongoTransactional
    @Override
    public SignInResVo signIn(String userId, Integer isDouble, List<String> tmplIds)
    throws Exception {
        String lockId = null;
        String lockKey = RedisKey.LOCK_REDIS_PREFIX_GOLDCOIN + userId;
        String lockIdProp = null;
        String lockKeyProp = RedisKey.LOCK_REDIS_PREFIX_PROP + userId;
        try {
            lockId = redisLock.lockNonNull(lockKey);
            lockIdProp = redisLock.lockNonNull(lockKeyProp);
            LocalDate today = LocalDate.now();
            int dayOfWeek = today.getDayOfWeek().getValue();

```

```

        int week = DateUtil.getYearWeek(today);
        isDouble = isDouble == null ? 0 : isDouble;
        SignInSettings signInSettings = mongoCommonDao.findByProperty("dayOfWeek",
dayOfWeek, SignInSettings.class);
        SignInDailyRecord signInDailyRecord = new SignInDailyRecord();
        signInDailyRecord.setId(IdUtil.nextId());
        signInDailyRecord.setCreatedAt(LocalDateTime.now());
        signInDailyRecord.setUserId(userId);
        signInDailyRecord.setSignInDate(today);
        signInDailyRecord.setDayOfWeek(dayOfWeek);
        signInDailyRecord.setWeek(week);
        signInDailyRecord.setTitle(signInSettings.getTitle());
        signInDailyRecord.setAwardType(signInSettings.getAwardType());
        String awardDesc = signInSettings.getAwardDesc();
        if (isDouble > 0) {
            signInDailyRecord.setAwardValue(signInSettings.getAwardValue() * 2);
            signInDailyRecord.setIsDouble(1);
            awardDesc = signInSettings.getAwardDesc2();
        } else {
            signInDailyRecord.setAwardValue(signInSettings.getAwardValue());
            signInDailyRecord.setIsDouble(0);
        }
        signInDailyRecordDao.insert(signInDailyRecord);
        UserStatis userStatis = userStatisDao.findById(userId);
        List<String> updateProperties = new ArrayList<>();
        userStatis.setSignInCount(userStatis.getSignInCount() + 1);
        updateProperties.add("signInCount");
        //奖励的类型, 10:金币; 3:复活道具
        if
(EnumSignAwardType.GOLD_COIN_10.getCode().equals(signInSettings.getAwardType())) { //10:
金币
            GoldCoinRecord goldCoinRecord =
goldCoinRecordDao.saveGoldCoinRecord(userId,
signInDailyRecord.getAwardValue(), signInDailyRecord.getId(),
EnumGoldCoinRecordType.SIGNIN_17);
            userStatis.setUsableGoldCoin(goldCoinRecord.getUsableGoldCoin());
            updateProperties.add("usableGoldCoin");
        } else if
(EnumSignAwardType.PROP_HALF_1.getCode().equals(signInSettings.getAwardType())) { //1:
去除 50%道具
            PropRecord propRelive = propRecordDao.savePropRecord(userId,
signInDailyRecord.getAwardValue(),
signInDailyRecord.getId(), EnumPropType.PROP_HALF_1,
EnumPropRecordType.USE_HALF_11);
            userStatis.setUsableReliveProp(propRelive.getUsableProp());
            updateProperties.add("usableHalfProp");
        } else if
(EnumSignAwardType.PROP_RIGHT_2.getCode().equals(signInSettings.getAwardType())) { //2:
正确答案道具
            PropRecord propRelive = propRecordDao.savePropRecord(userId,

```

```

signInDailyRecord.getAwardValue(),
                signInDailyRecord.getId(),                EnumPropType.PROP_RIGHT_2,
EnumPropRecordType.USE_RIGHT_12);
                userStatis.setUsableReliveProp(propRelive.getUsableProp());
                updateProperties.add("usableRightProp");
            }
            else
            if
(EnumSignAwardType.PROP_RELIVE_3.getCode().equals(signInSettings.getAwardType())) { //3:
复活道具
                PropRecord propRelive = propRecordDao.savePropRecord(userId,
signInDailyRecord.getAwardValue(),
                signInDailyRecord.getId(),                EnumPropType.PROP_RELIVE_3,
EnumPropRecordType.SIGNIN_RELIVE_3);
                userStatis.setUsableReliveProp(propRelive.getUsableProp());
                updateProperties.add("usableReliveProp");
            }
            userStatisDao.updateProperties(userStatis,
ArrayUtil.toArray(updateProperties, String.class));
            Update update = new Update().set("lastSignInTime", LocalDateTime.now());
            if(tmplIds != null && tmplIds.contains(TnzbConst.SIGNIN_TMPLID)) {
                update.inc("signInTplCount", 1);
            }
            userDao.updateFirst(MongoUtil.getQueryById(userId), update);
            return
            new
SignInResVo().setIsSuccess(1).setAwardDesc(awardDesc).setIcon(signInSettings.getIcon()
)
                .setUsableGoldCoin(userStatis.getUsableGoldCoin()).setUsableHalfPr
op(userStatis.getUsableHalfProp())
                .setUsableRightProp(userStatis.getUsableRightProp()).setUsableReli
veProp(userStatis.getUsableReliveProp());
        } finally {
            if (StrUtil.isNotBlank(lockId)) {
                redisLock.unlock(lockKey, lockId);
            }
            if (StrUtil.isNotBlank(lockIdProp)) {
                redisLock.unlock(lockKeyProp, lockIdProp);
            }
        }
    }

    private SignInDailyRecord getSignInDailyRecord(List<SignInDailyRecord>
signInDailyRecordList, int dayOfWeek) {
        for (SignInDailyRecord signInDailyRecord : signInDailyRecordList) {
            if(signInDailyRecord.getDayOfWeek() == dayOfWeek) {
                return signInDailyRecord;
            }
        }
        return null;
    }
}

package com.olal00.tnzb.app.task.service;
import cn.hutool.core.util.ArrayUtil;

```

```
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.app.task.form.vo.TaskResVo;
import com.olal00.tnzb.app.task.form.vo.TaskVo;
import com.olal00.tnzb.common.config.MongoTransactional;
import com.olal00.tnzb.common.enums.*;
import com.olal00.tnzb.common.exception.MsgException;
import com.olal00.tnzb.common.redis.RedisKey;
import com.olal00.tnzb.common.redis.RedisLock;
import com.olal00.tnzb.common.util.IdUtil;
import com.olal00.tnzb.data.dao.base.MongoCommonDao;
import com.olal00.tnzb.data.dao.common.GoldCoinRecordDao;
import com.olal00.tnzb.data.dao.common.PropRecordDao;
import com.olal00.tnzb.data.dao.common.UserStatisDao;
import com.olal00.tnzb.data.domain.common.*;
import com.olal00.tnzb.data.domain.form.TaskState;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
/**
@Service
public class TaskBaseServiceImpl extends BaseServiceImpl<TaskBase, String> implements
TaskBaseService {
    @Autowired
    private RedisLock redisLock;
    @Autowired
    private UserStatisDao userStatisDao;
    @Autowired
    private GoldCoinRecordDao goldCoinRecordDao;
    @Autowired
    private PropRecordDao propRecordDao;
    @Autowired
    private MongoCommonDao<TaskRecord, String> taskRecordDao;
    @Override
    public List<TaskVo> findTasks(String userId) throws Exception {
        return buildTaskVos(userId, false);
    }
    //任务列表成功观看广告
    @Override
    public List<TaskVo> viewTaskAd(String userId) throws Exception {
        return buildTaskVos(userId, true);
    }
    private List<TaskVo> buildTaskVos(String userId, boolean isViewAd) throws Exception
{
```

```

List<TaskBase> taskBaseList =
mongoCommonDao.findAll(Sort.by(Sort.Order.asc("orderNum")), TaskBase.class);
List<TaskVo> voList = new ArrayList<>(taskBaseList.size());
UserStatis userStatis = userStatisDao.findById(userId);
TaskState taskState = userStatis.getTaskState();
if(taskState == null) {
    taskState = new TaskState();
}
if(isViewAd) {
    LocalDateTime now = LocalDateTime.now();
    if(taskState.getViewAdTime() != null &&
taskState.getViewAdTime().plusSeconds(3).isAfter(now)) {
        throw new MsgException("请勿重复提交");
    }
    taskState.setTodayViewAdCount(taskState.getTodayViewAdCount() + 1);
    taskState.setViewAdTime(now); //这个必须放在 setTodayViewAdCount 后面
}
LocalDateTime today = LocalDateTime.of(LocalDate.now(), LocalTime.MIN);
for (TaskBase taskBase : taskBaseList) {
    TaskVo vo = new TaskVo();
    vo.setTaskId(taskBase.getId());
    vo.setIcon(taskBase.getIcon());
    vo.setTitle(taskBase.getTitle());
    vo.setAwardDesc(taskBase.getAwardDesc());
    vo.setTaskType(taskBase.getTaskType());
    if(EnumTaskType.DAY_OPEN_1.getCode().equals(taskBase.getTaskType())) {
        if(taskState.getOpenQualifyingAwardTime() != null &&
taskState.getOpenQualifyingAwardTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_YFF_3.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_YFF_3.getText());
        } else if(taskState.getOpenQualifyingTime() != null &&
taskState.getOpenQualifyingTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_DLQ_2.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_DLQ_2.getText());
        } else {
            vo.setStatus(EnumTaskStatus.STATUS_ZRW_1.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_ZRW_1.getText());
        }
    } else if(EnumTaskType.DAY_AD_2.getCode().equals(taskBase.getTaskType())) {
        if(taskState.getViewAdArardTime() != null &&
taskState.getViewAdArardTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_YFF_3.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_YFF_3.getText());
        } else if(taskState.getViewAdTime() != null &&
taskState.getViewAdTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_DLQ_2.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_DLQ_2.getText());
        } else {
            vo.setStatus(EnumTaskStatus.STATUS_ZRW_1.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_ZRW_1.getText());
        }
    }
}

```



```

        }
        if(isViewAd && taskState.getTodayViewAdCount() == 1) {
            userStatis.setTaskCount(userStatis.getTaskCount() + 1);
        }
    } else if(EnumTaskType.DAY_ADS_3.getCode().equals(taskBase.getTaskType()))
    {
        if(taskState.getViewAdsArardTime() != null &&
taskState.getViewAdsArardTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_YFF_3.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_YFF_3.getText());
        } else if(taskState.getTodayViewAdCount() >= taskBase.getNeedCount()) {
            vo.setStatus(EnumTaskStatus.STATUS_DLQ_2.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_DLQ_2.getText());
        } else {
            vo.setStatus(EnumTaskStatus.STATUS_ZRW_1.getCode());
            //vo.setStatusText(EnumTaskStatus.STATUS_ZRW_1.getText());
            vo.setStatusText(taskState.getTodayViewAdCount() + "/" +
taskBase.getNeedCount());
        }
        if(isViewAd && Objects.equals(taskState.getTodayViewAdCount(),
taskBase.getNeedCount())) {
            userStatis.setTaskCount(userStatis.getTaskCount() + 1);
        }
    } else if(EnumTaskType.DAY_WIN_4.getCode().equals(taskBase.getTaskType()))
    {
        if(taskState.getWinQualifyingAwardTime() != null &&
taskState.getWinQualifyingAwardTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_YFF_3.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_YFF_3.getText());
        } else if(taskState.getWinQualifyingTime() != null &&
taskState.getWinQualifyingTime().isAfter(today)) {
            vo.setStatus(EnumTaskStatus.STATUS_DLQ_2.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_DLQ_2.getText());
        } else {
            vo.setStatus(EnumTaskStatus.STATUS_ZRW_1.getCode());
            vo.setStatusText(EnumTaskStatus.STATUS_ZRW_1.getText());
        }
    } else {
        continue;
    }
    voList.add(vo);
}
if(isViewAd) {
    userStatis.setTaskState(taskState);
    userStatisDao.updateProperties(userStatis, "taskState", "taskCount");
}
return voList;
}
@Transactional
@Override

```

```

public TaskResVo receiveAward(String userId, String taskId) throws Exception {
    String lockId = null;
    String lockKey = RedisKey.LOCK_REDIS_PREFIX_GOLDCOIN + userId;
    String lockIdProp = null;
    String lockKeyProp = RedisKey.LOCK_REDIS_PREFIX_PROP + userId;
    try {
        lockId = redisLock.lockNonNull(lockKey);
        lockIdProp = redisLock.lockNonNull(lockKeyProp);
        LocalDateTime today = LocalDateTime.of(LocalDate.now(), LocalTime.MIN);
        LocalDateTime now = LocalDateTime.now();
        TaskBase taskBase = mongoCommonDao.findById(taskId, TaskBase.class);
        UserStatis userStatis = userStatisDao.findById(userId);
        TaskState taskState = userStatis.getTaskState();
        List<String> updateProperties = new ArrayList<>();
        String msg = "任务不是待领取状态";
        if(taskState == null) {
            throw new MsgException(msg);
        } else if(EnumTaskType.DAY_OPEN_1.getCode().equals(taskBase.getTaskType()))
        {
            if(taskState.getOpenQualifyingAwardTime() != null &&
taskState.getOpenQualifyingAwardTime().isAfter(today)) {
                throw new MsgException(msg);
            } else if(taskState.getOpenQualifyingTime() != null &&
taskState.getOpenQualifyingTime().isAfter(today)) {
                taskState.setOpenQualifyingAwardTime(now);
            } else {
                throw new MsgException(msg);
            }
        } else if(EnumTaskType.DAY_AD_2.getCode().equals(taskBase.getTaskType())) {
            if(taskState.getViewAdArardTime() != null &&
taskState.getViewAdArardTime().isAfter(today)) {
                throw new MsgException(msg);
            } else if(taskState.getViewAdTime() != null &&
taskState.getViewAdTime().isAfter(today)) {
                taskState.setViewAdArardTime(now);
            } else {
                throw new MsgException(msg);
            }
        } else if(EnumTaskType.DAY_ADS_3.getCode().equals(taskBase.getTaskType()))
        {
            if(taskState.getViewAdsArardTime() != null &&
taskState.getViewAdsArardTime().isAfter(today)) {
                throw new MsgException(msg);
            } else if(taskState.getTodayViewAdCount() >= taskBase.getNeedCount()) {
                taskState.setViewAdsArardTime(now);
            } else {
                throw new MsgException(msg);
            }
        } else if(EnumTaskType.DAY_WIN_4.getCode().equals(taskBase.getTaskType()))
        {

```

```

        if(taskState.getWinQualifyingAwardTime() != null &&
taskState.getWinQualifyingAwardTime().isAfter(today)) {
            throw new MsgException(msg);
        } else if(taskState.getWinQualifyingTime() != null &&
taskState.getWinQualifyingTime().isAfter(today)) {
            taskState.setWinQualifyingAwardTime(now);
        } else {
            throw new MsgException(msg);
        }
    } else {
        throw new MsgException(msg);
    }
    TaskRecord taskRecord = new TaskRecord();
    taskRecord.setId(IdUtil.nextId());
    taskRecord.setCreatedAt(LocalDateTime.now());
    taskRecord.setUserId(userId);
    taskRecord.setTaskDate(LocalDate.now());
    taskRecord.setTaskId(taskId);
    taskRecord.setTitle(taskBase.getTitle());
    taskRecord.setAwardType(taskBase.getAwardType());
    taskRecord.setAwardValue(taskBase.getAwardValue());
    taskRecordDao.insert(taskRecord);
    userStatis.setTaskState(taskState);
    updateProperties.add("taskState");
    userStatis.setTaskAwardCount(userStatis.getTaskAwardCount() + 1);
    updateProperties.add("taskAwardCount");
    //奖励的类型, 10:金币; 1:去除 50%道具
    if
(EnumSignAwardType.GOLD_COIN_10.getCode().equals(taskBase.getAwardType())) { //10:金币
        GoldCoinRecord goldCoinRecord =
goldCoinRecordDao.saveGoldCoinRecord(userId,
            taskRecord.getAwardValue(), taskRecord.getId(),
EnumGoldCoinRecordType.SIGNIN_17);
        userStatis.setUsableGoldCoin(goldCoinRecord.getUsableGoldCoin());
        updateProperties.add("usableGoldCoin");
    } else if
(EnumSignAwardType.PROP_HALF_1.getCode().equals(taskBase.getAwardType())) { //1:去除 50%
道具
        PropRecord propRelive = propRecordDao.savePropRecord(userId,
taskRecord.getAwardValue(),
            taskRecord.getId(), EnumPropType.PROP_HALF_1,
EnumPropRecordType.USE_HALF_11);
        userStatis.setUsableReliveProp(propRelive.getUsableProp());
        updateProperties.add("usableHalfProp");
    } else if
(EnumSignAwardType.PROP_RIGHT_2.getCode().equals(taskBase.getAwardType())) { //2:正确答
案道具
        PropRecord propRelive = propRecordDao.savePropRecord(userId,
taskRecord.getAwardValue(),
            taskRecord.getId(), EnumPropType.PROP_RIGHT_2,

```

```

EnumPropRecordType.USE_RIGHT_12);
        userStatis.setUsableReliveProp(propRelive.getUsableProp());
        updateProperties.add("usableRightProp");
    }
    else
    {
        if
        (EnumSignAwardType.PROP_RELIVE_3.getCode().equals(taskBase.getAwardType())) { //3:复活
        道具
            PropRecord propRelive = propRecordDao.savePropRecord(userId,
            taskRecord.getAwardValue(),
            taskRecord.getId(),
            EnumPropType.PROP_RELIVE_3,
            EnumPropRecordType.USE_RELIVE_13);
            userStatis.setUsableReliveProp(propRelive.getUsableProp());
            updateProperties.add("usableReliveProp");
        }
        userStatisDao.updateProperties(userStatis,
        ArrayUtil.toArray(updateProperties, String.class));
        return
        new
        TaskResVo().setIsSuccess(1).setStatus(EnumTaskStatus.STATUS_YFF_3.getCode())
            .setStatusText(EnumTaskStatus.STATUS_YFF_3.getText())
            .setAwardDesc(taskBase.getAwardDesc2()).setIcon(taskBase.getAwardI
            con())
            .setUsableGoldCoin(userStatis.getUsableGoldCoin()).setUsableHalfPr
            op(userStatis.getUsableHalfProp())
            .setUsableRightProp(userStatis.getUsableRightProp()).setUsableReli
            veProp(userStatis.getUsableReliveProp());
    } finally {
        if (StrUtil.isNotBlank(lockId)) {
            redisLock.unlock(lockKey, lockId);
        }
        if (StrUtil.isNotBlank(lockIdProp)) {
            redisLock.unlock(lockKeyProp, lockIdProp);
        }
    }
}
}

package com.olal00.tnzb.app.question.form.vo;
import com.olal00.tnzb.common.TnzbConst;
import lombok.Data;
import lombok.experimental.Accessors;
import java.io.Serializable;
/**
@Data
@Accessors(chain = true)
public class UpLevelVo implements Serializable {
    private static final long serialVersionUID = TnzbConst.SERIAL_VERSION_UID;
    private Integer levelCount; //一共多少关
    private Integer level; //当前在第几关
    private Double star; //当前关得了几颗星
    private Integer needPower = TnzbConst.POWER_USE; //需要精力值
    private Integer usablePower; //精力值
    private Integer power; //奖励精力值

```

```

        private Integer nextPowerSeconnds; //下次加精力值还需多少秒(精力值满的时候该值是 0)
    }

package com.olal00.tnzb.app.question.service;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.app.question.form.req.UpLevelReq;
import com.olal00.tnzb.app.question.form.vo.UpLevelVo;
import com.olal00.tnzb.common.TnzbConst;
import com.olal00.tnzb.common.config.MongoTransactional;
import com.olal00.tnzb.data.dao.MongoUtil;
import com.olal00.tnzb.data.dao.common.UserStatisDao;
import com.olal00.tnzb.data.domain.common.UpLevelLog;
import com.olal00.tnzb.data.domain.common.UserStatis;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.time.Duration;
import java.time.LocalDateTime;
/**
@Service
public class UpLevelLogServiceImpl extends BaseServiceImpl<UpLevelLog, String> implements
UpLevelLogService {
    @Autowired
    private UserStatisDao userStatisDao;
    @Override
    public UpLevelLog findLastLevel(String userId) throws Exception {
        return
mongoCommonDao.findOne(MongoUtil.getQuery().addCriteria(Criteria.where("userId").is(us
erId))
                .with(Sort.by(Sort.Order.desc("level"),      Sort.Order.desc("star"))),
UpLevelLog.class);
    }
    @MongoTransactional
    @Override
    public UpLevelVo saveUserAnswerLog(UpLevelReq req) throws Exception {
        UpLevelVo vo = new UpLevelVo();
        UserStatis userStatis = userStatisDao.findById(req.getUserId());
        UpLevelLog upLevelLog = req.toUserAnswerLog();
        mongoCommonDao.insert(upLevelLog);
        LocalDateTime now = upLevelLog.getCreatedAt();
        boolean update = false;
        int usablePower = userStatis.getUsablePower();
        if(upLevelLog.getPower() > 0) {
            update = true;
            usablePower += upLevelLog.getPower();
            if(usablePower > TnzbConst.POWER_MAX_SIZE) {
                usablePower = TnzbConst.POWER_MAX_SIZE;
                userStatis.setLastAddPowerTime(now);
            }
        }
    }
}

```

```

        int nextPowerSeconnds = 0;
        if(usablePower < TnzbConst.POWER_MAX_SIZE) {
            int seconds = (int) Duration.between(userStatis.getLastAddPowerTime(),
now).getSeconds();
            if(seconds < TnzbConst.POWER_INTERVAL_SECONDS) { //还没到间隔时间
                nextPowerSeconnds = TnzbConst.POWER_INTERVAL_SECONDS - seconds;
            } else {
                update = true;
                int passSeconds = seconds % TnzbConst.POWER_INTERVAL_SECONDS;

userStatis.setLastAddPowerTime(LocalDateTime.now().minusSeconds(passSeconds));
                nextPowerSeconnds = TnzbConst.POWER_INTERVAL_SECONDS - passSeconds;
                usablePower = usablePower + seconds / TnzbConst.POWER_INTERVAL_SECONDS *
TnzbConst.POWER_STEP;
                if(usablePower > TnzbConst.POWER_MAX_SIZE) {
                    usablePower = TnzbConst.POWER_MAX_SIZE;
                    userStatis.setLastAddPowerTime(now);
                    nextPowerSeconnds = 0;
                }
            }
        }
        if(update) {
            userStatisDao.updateProperties(userStatis.setUsablePower(usablePower),
                "usablePower", "lastAddPowerTime");
        }

vo.setPower(upLevelLog.getPower()).setUsablePower(usablePower).setNextPowerSeconnds(nextPowerSeconnds);
        return vo.setLevel(upLevelLog.getLevel()).setStar(upLevelLog.getStar());
    }
}

package com.olal00.tnzb.app.book.service;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.app.book.form.req.BookLevelReq;
import com.olal00.tnzb.app.book.form.vo.BookLevelVo;
import com.olal00.tnzb.common.TnzbConst;
import com.olal00.tnzb.common.config.MongoTransactional;
import com.olal00.tnzb.data.dao.MongoUtil;
import com.olal00.tnzb.data.dao.common.UserStatisDao;
import com.olal00.tnzb.data.domain.common.BookLevelLog;
import com.olal00.tnzb.data.domain.common.UserStatis;
import com.olal00.tnzb.data.domain.form.BookCatalog;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.time.Duration;
import java.time.LocalDateTime;
/**
@Service

```

```

public class BookLevelLogServiceImpl extends BaseServiceImpl<BookLevelLog, String>
implements BookLevelLogService {
    @Autowired
    private UserStatisDao userStatisDao;
    @Autowired
    private BookService bookService;
    @Override
    public BookLevelLog findLastLevel(String userId, String chapterId) throws Exception
    {
        return
        mongoCommonDao.findOne(MongoUtil.getQuery().addCriteria(Criteria.where("userId").is(us
erId)
                                .and("bookCatalog.chapterId").is(chapterId))
                                .with(Sort.by(Sort.Order.desc("level"), Sort.Order.desc("star"))),
BookLevelLog.class);
    }
    @MongoTransactional
    @Override
    public BookLevelVo saveUserAnswerLog(BookLevelReq req) throws Exception {
        BookLevelVo vo = new BookLevelVo();
        UserStatis userStatis = userStatisDao.findById(req.getUserId());
        BookCatalog bookCatalog = bookService.getBookCatalog(req.getChapterId());
        BookLevelLog bookLevelLog = req.toUserAnswerLog(bookCatalog);
        mongoCommonDao.insert(bookLevelLog);
        LocalDateTime now = bookLevelLog.getCreatedAt();
        boolean update = false;
        int usablePower = userStatis.getUsablePower();
        if(bookLevelLog.getPower() > 0) {
            update = true;
            usablePower += bookLevelLog.getPower();
            if(usablePower > TnzbConst.POWER_MAX_SIZE) {
                usablePower = TnzbConst.POWER_MAX_SIZE;
                userStatis.setLastAddPowerTime(now);
            }
        }
        int nextPowerSeconnds = 0;
        if(usablePower < TnzbConst.POWER_MAX_SIZE) {
            int seconds = (int) Duration.between(userStatis.getLastAddPowerTime(),
now).getSeconds();
            if(seconds < TnzbConst.POWER_INTERVAL_SECONDS) { //还没到间隔时间
                nextPowerSeconnds = TnzbConst.POWER_INTERVAL_SECONDS - seconds;
            } else {
                update = true;
                int passSeconds = seconds % TnzbConst.POWER_INTERVAL_SECONDS;

                userStatis.setLastAddPowerTime(LocalDateTime.now().minusSeconds(passSeconds));
                nextPowerSeconnds = TnzbConst.POWER_INTERVAL_SECONDS - passSeconds;
                usablePower = usablePower + seconds / TnzbConst.POWER_INTERVAL_SECONDS *
TnzbConst.POWER_STEP;
                if(usablePower > TnzbConst.POWER_MAX_SIZE) {

```

```

        usablePower = TnzbConst.POWER_MAX_SIZE;
        userStatis.setLastAddPowerTime(now);
        nextPowerSeconnds = 0;
    }
}
}
if(update) {
    userStatisDao.updateProperties(userStatis.setUsablePower(usablePower),
        "usablePower", "lastAddPowerTime");
}

vo.setPower(bookLevelLog.getPower()).setUsablePower(usablePower).setNextPowerSeconnds(
nextPowerSeconnds);
    return
vo.setLevel(bookLevelLog.getLevel()).setStar(bookLevelLog.getStar()).setBookCatalog(bookCatalog);
}
}

package com.olal00.tnzb.app.book.service;
import cn.hutool.core.collection.CollectionUtil;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.app.question.form.vo.QuestionGroupVo;
import com.olal00.tnzb.data.dao.MongoUtil;
import com.olal00.tnzb.data.domain.common.BookQuestion;
import com.olal00.tnzb.data.domain.form.BookCatalog;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.util.ArrayList;
import java.util.List;
/**
@Service
public class BookQuestionServiceImpl extends BaseServiceImpl<BookQuestion, String>
implements BookQuestionService {
    @Override
    public QuestionGroupVo getQuestionGroup(String chapterId, Integer level) throws
Exception {
        QuestionGroupVo questionGroupVo = new QuestionGroupVo();
        questionGroupVo.setLevel(level);
        String groupId = chapterId + "_" + level;
        questionGroupVo.setQuestionGroupId(groupId);
        List<BookQuestion> questionList =
mongoCommonDao.find(MongoUtil.getQuery().addCriteria(

Criteria.where("groupId").is(groupId)).with(Sort.by(Sort.Order.asc("orderNum"))),
BookQuestion.class);
        if(CollectionUtil.isEmpty(questionList)) {
            return questionGroupVo;
        }
        BookCatalog bookCatalog = questionList.get(0).getBookCatalog();

```



```

        questionGroupVo.setBookCatalog(bookCatalog);
        List<QuestionGroupVo.QuestionVo> questionVoList = new
ArrayList<>(questionList.size());
        for (int i = 0; i < questionList.size(); i++) {
            questionVoList.add(QuestionGroupVo.QuestionVo.of(questionList.get(i)));
        }
        questionGroupVo.setQuestions(questionVoList);
        return questionGroupVo;
    }

    @Override
    public int levelCount(String chapterId) throws Exception {
        BookQuestion lastOne = mongoCommonDao.findOne(MongoUtil.getQuery().addCriteria(
            Criteria.where("bookCatalog.chapterId").is(chapterId))
                .with(Sort.by(Sort.Order.desc("groupOrderNum"))),
BookQuestion.class);
        return lastOne == null ? 0 : lastOne.getGroupOrderNum();
    }
}

package com.olal00.tnzb.app.book.service;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.app.base.BaseServiceImpl;
import com.olal00.tnzb.data.dao.MongoUtil;
import com.olal00.tnzb.data.domain.common.Book;
import com.olal00.tnzb.data.domain.form.BookCatalog;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.stereotype.Service;
import java.util.List;

/**
@Service
public class BookServiceImpl extends BaseServiceImpl<Book, String> implements BookService
{
    @Override
    public List<Book> findBySubjectId(Integer subjectId) throws Exception {
        return mongoCommonDao.find(MongoUtil.getQuery("name", "shortName").addCriteria(
Criteria.where("subjectId").is(subjectId).and("exerciseStatus").is("ONLINE").and("leve
l").is(1))
                .with(Sort.by(Sort.Order.asc("rank"))), Book.class);
    }

    @Override
    public List<Book> findByParent(String parentId) throws Exception {
        return mongoCommonDao.find(MongoUtil.getQuery("name", "shortName").addCriteria(
Criteria.where("parentId").is(parentId).and("exerciseStatus").is("ONLINE"))
                .with(Sort.by(Sort.Order.asc("rank"))), Book.class);
    }

    @Override
    public BookCatalog getBookCatalog(String chapterId) throws Exception {
        BookCatalog bookCatalog = new BookCatalog();

```

```
        bookCatalog.setChapterId(chapterId);
        Book chapter = mongoCommonDao.findById(chapterId, Book.class);
        if(chapter == null || StrUtil.isBlank(chapter.getParentId())) {
            return bookCatalog;
        }
        bookCatalog.setChapterName(chapter.getName());
        Book book = mongoCommonDao.findById(chapter.getParentId(), Book.class);
        if(book == null) {
            return bookCatalog;
        }
        bookCatalog.setSubjectId(book.getSubjectId());
        bookCatalog.setSubjectLevel(book.getSubjectLevel());
        bookCatalog.setGrade(book.getGrade());
        bookCatalog.setSeriesId(book.getParentId());
        bookCatalog.setSeriesName(book.getParentName());
        bookCatalog.setBookId(book.getId());
        bookCatalog.setBookName(book.getName());
        bookCatalog.setShortName(book.getShortName());
        return bookCatalog;
    }
}

package com.olal00.tnzb.app.book.web;
import cn.hutool.core.util.StrUtil;
import com.olal00.tnzb.app.book.form.req.BookLevelReq;
import com.olal00.tnzb.app.book.service.BookService;
import com.olal00.tnzb.common.config.VlRestController;
import com.olal00.tnzb.common.response.Ret;
import com.olal00.tnzb.common.response.RetUtil;
import com.olal00.tnzb.data.domain.common.Book;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import java.util.List;
/**
@VlRestController("/book")
public class BookController {
    @Autowired
    private BookService bookService;
    @GetMapping("/findBySubjectId")
    public Ret<List<Book>> findBySubjectId(BookLevelReq req) throws Exception {
        if(req.getSubjectId() == null) {
            return RetUtil.fail("请选择学科");
        }
        return RetUtil.ok(bookService.findBySubjectId(req.getSubjectId()));
    }
    @GetMapping("/findByParent")
    public Ret<List<Book>> findByParent(BookLevelReq req) throws Exception {
        if(StrUtil.isBlank(req.getParentId())) {
            return RetUtil.fail("请选择父节点");
        }
        return RetUtil.ok(bookService.findByParent(req.getParentId()));
    }
}
```

```
}  
}  
  
package com.olal00.tnzb.app.book.web;  
import cn.hutool.core.collection.CollectionUtil;  
import cn.hutool.core.util.StrUtil;  
import com.olal00.tnzb.app.book.form.req.BookLevelReq;  
import com.olal00.tnzb.app.book.form.vo.BookLevelVo;  
import com.olal00.tnzb.app.book.service.BookLevelLogService;  
import com.olal00.tnzb.app.book.service.BookQuestionService;  
import com.olal00.tnzb.app.book.service.BookService;  
import com.olal00.tnzb.app.question.form.vo.QuestionGroupVo;  
import com.olal00.tnzb.app.user.service.UserStatisService;  
import com.olal00.tnzb.common.TnzbConst;  
import com.olal00.tnzb.common.config.VIRestController;  
import com.olal00.tnzb.common.response.Ret;  
import com.olal00.tnzb.common.response.RetUtil;  
import com.olal00.tnzb.data.domain.common.BookLevelLog;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PostMapping;  
import org.springframework.web.bind.annotation.RequestBody;  
/**  
@VIRestController("/bookLevel")  
public class BookLevelController {  
    @Autowired  
    private BookQuestionService bookQuestionService;  
    @Autowired  
    private BookLevelLogService bookLevelLogService;  
    @Autowired  
    private UserStatisService userStatisService;  
    @Autowired  
    private BookService bookService;  
    //用户闯关信息  
    @GetMapping("/userLevel")  
    public Ret<BookLevelVo> userLevel(BookLevelReq req) throws Exception {  
        if(StrUtil.isBlank(req.getChapterId())) {  
            return RetUtil.fail("请选择章节");  
        }  
        int levelCount = bookQuestionService.levelCount(req.getChapterId());  
        BookLevelVo vo = new BookLevelVo();  
        vo.setLevelCount(levelCount);  
        if(levelCount == 0) {  
            return  
RetUtil.ok(vo.setLevel(0).setStar(0.0).setBookCatalog(bookService.getBookCatalog(req.g  
etChapterId())));  
        }  
        BookLevelLog bookLevelLog = bookLevelLogService.findLastLevel(req.getUserId(),  
req.getChapterId());  
        if(bookLevelLog == null || bookLevelLog.getLevel() == null) {  
            vo.setLevel(1);  
        }  
    }  
}
```

```

        vo.setStar(0.0);
        vo.setBookCatalog(bookService.getBookCatalog(req.getChapterId()));
    } else {
        vo.setBookCatalog(bookLevelLog.getBookCatalog());
        if(bookLevelLog.getStar() > 2.9 && bookLevelLog.getLevel() <
vo.getLevelCount()) {
            vo.setLevel(bookLevelLog.getLevel() + 1);
            vo.setStar(0.0);
        } else {
            vo.setLevel(bookLevelLog.getLevel());
            vo.setStar(bookLevelLog.getStar());
        }
    }
    return RetUtil.ok(vo);
}
//加载题目资源
@GetMapping("/getQuestionGroup")
public Ret<QuestionGroupVo> getQuestionGroup(BookLevelReq req) throws Exception {
    if(StrUtil.isBlank(req.getChapterId())) {
        return RetUtil.fail("请选择章节");
    }
    userStatisService.usePower(req.getUserId(), TnzbConst.POWER_USE);
    return RetUtil.ok(bookQuestionService.getQuestionGroup(req.getChapterId(),
req.getLevel()));
}
//上报答题结果
@PostMapping("/saveUserAnswerLog")
public Ret<BookLevelVo> saveUserAnswerLog(@RequestBody BookLevelReq req) throws
Exception {
    if(StrUtil.isBlank(req.getChapterId())) {
        return RetUtil.fail("请选择章节");
    }
    if(req.getLevel() == null || req.getLevel() < 1) {
        return RetUtil.fail("参数错误");
    }
    if(CollectionUtil.isEmpty(req.getUserAnswers())) {
        return RetUtil.fail("缺少答题结果");
    }
    for (BookLevelReq.UserAnswer answer : req.getUserAnswers()) {
        if(answer == null || StrUtil.isBlank(answer.getQuestionId()) ||
answer.getIsSuper() == null
            || answer.getAnswerIndex() == null) {
            return RetUtil.fail("缺少答题结果");
        }
    }
    return
RetUtil.ok(bookLevelLogService.saveUserAnswerLog(req).setLevelCount(bookQuestionServic
e.levelCount(req.getChapterId())));
}
}

```